


UNIVERSIDAD MADERO PUEBLA
INCORPORADA A LA SEP

# Desarrollo de una app web de un tutor virtual UMAD

QUE PARA OBTENER EL TÍTULO DE

Licenciado en Ingeniería de Software

Presentan:

Emmanuel Baltazar López

Oscar Juárez Ortiz

Pedro Emiliano Romero Osorio

Director de tesis

Carlos Zapata Bretón

"Nuevas respuestas para nuevos tiempos"

2026

# Resumen

El presente trabajo describe el desarrollo de una plataforma web de tutoría virtual impulsada por inteligencia artificial, diseñada para la Universidad Madero Puebla (UMAD), orientada específicamente a los estudiantes de la carrera de Ingeniería de Software. La problemática central identificada radica en la dificultad que enfrentan los alumnos para acceder a información confiable y oportuna que refuerce los contenidos académicos revisados en clase, lo que deriva en rezago, bajo aprovechamiento y reprobación de asignaturas clave.

Como solución, se desarrolló una aplicación web interactiva que integra un chatbot de inteligencia artificial generativa (Google Gemini 2.5), un avatar 2D reactivo con síntesis de voz nativa (Web Speech API) y un sistema de autenticación restringido a correos institucionales bajo el dominio @umad.edu.mx. El backend fue construido con Node.js y Express, la persistencia de datos se gestiona mediante SQLite, y el frontend se desarrolló con HTML5, CSS3 y JavaScript vainilla. La metodología adoptada fue el Desarrollo Rápido de Aplicaciones (RAD), lo que permitió la construcción iterativa y la integración ágil de tecnologías emergentes.

El sistema implementa doce requerimientos funcionales y cinco no funcionales, cubriendo módulos de autenticación, gestión de sesiones de IA, síntesis de voz y almacenamiento cronológico del historial de consultas. El resultado es un tutor virtual especializado en el plan de estudios de Ingeniería de Software, capaz de proporcionar orientación académica personalizada, mejorando la experiencia de aprendizaje de los estudiantes de la UMAD.

# Índice general

**Resumen** III

**Índice de figuras** VIII

**Índice de tablas** X

**1. 1. Introducción** 1

1.1. Planteamiento del problema . . . . . 3

1.2. Objetivo general . . . . . 4

1.3. Objetivos específicos . . . . . 4

1.4. Alcance y limitaciones . . . . . 5

1.4.1. Alcance . . . . . 5

1.4.2. Limitaciones . . . . . 6

1.5. Justificación . . . . . 7

**2. 2. Marco teórico** **9**

2.1. Sistemas de Tutoría Inteligente (STI) basados en LLM . . . . . 10

2.2. Ingeniería de Prompts (Prompt Engineering) . . . . . 11

2.3. Tecnologías de Interacción Multimodal Web . . . . . 12

2.4. Aprendizaje Adaptativo . . . . . 13

2.5. Ética, Gobernanza y Uso Responsable de IA en Educación . . . . 14

2.6. Experiencia de Usuario y Diseño de Interacción en Sistemas Conversacionales Educativos . . . . . 15

2.7. Mapa Conceptual del Marco Teórico . . . . . 16

**3. 3. Desarrollo** **17**

3.1. Requerimientos . . . . . 17

3.1.1. Requerimientos Funcionales . . . . . 17

3.1.2. Requerimientos No Funcionales . . . . . 20

3.2. Planeación . . . . . 21

3.2.1. Plan de configuración . . . . . 21

3.2.2. Gantt . . . . . 25

3.3. Metodología . . . . . 25

3.3.1. Listado de Requerimientos . . . . . 26

3.3.2. Listado de formatos y reportes . . . . . 28

3.3.3. Diagrama de navegación . . . . . 30

3.3.4. Listado de objetos . . . . . 31

3.3.5. Diagrama de estados . . . . . 32

3.4. Análisis del Sistema . . . . . 33

3.4.1. Actores del Sistema . . . . . 33

3.4.2. Especificación de Casos de Uso Clave . . . . . 33

3.4.3. Diagrama de Flujo del Caso de Uso Principal . . . . . 35

3.5. Diseño Arquitectónico y de Datos . . . . . 36

3.5.1. Arquitectura del Sistema . . . . . 36

3.5.2. Modelo de Base de Datos Relacional . . . . . 37

**4. Conclusión y recomendaciones** **39**

4.1. Conclusión . . . . . 40

4.1.1. Métricas . . . . . 40

4.1.2. Conclusión . . . . . 43

4.1.3. Lecciones aprendidas . . . . . 44

4.2. Recomendaciones . . . . . 50

**Referencias** **62**

# Índice de figuras

2.1. Mapa Conceptual “Desarrollo del proyecto” . . . . . 16

3.1. Diagrama de Gantt “Desarrollo del proyecto” . . . . . 25

3.2. Diagrama “Navegación” . . . . . 30

3.3. Doagrama de clases “Listado de objetos” . . . . . 31

3.4. Diagrama “Estados” . . . . . 32

3.5. Diagrama de Flujo “Caso de uso principal” . . . . . 36

4.1. Esfuerzo técnico y entregables por integrante del equipo. . . . . 51

4.2. Esfuerzo técnico y entregables por integrante del equipo. . . . . 52

4.3. Tiempos de respuesta (latencia) por módulo del sistema. . . . . 53

4.4. Tiempos de respuesta (latencia) por módulo del sistema. . . . . 54

4.5. Curva de resolución de bugs a lo largo de las fases del proyecto. . 55

4.6. Curva de resolución de bugs a lo largo de las fases del proyecto. . 56

4.7. Tasa de aprobación del sistema por perfil de usuario simulado. . . 57

4.8. Tasa de aprobación del sistema por perfil de usuario simulado. . . 58

# Índice de tablas

3.1. Estructura de la tabla de Usuarios . . . . . 38

3.2. Estructura de la tabla de Interacciones . . . . . 38

4.1. Rendimiento y esfuerzo del equipo de desarrollo. . . . . 42

4.2. Tiempos de respuesta y tasa de éxito por módulo del sistema. . . . 42

4.3. Resolución de bugs y calidad de software por fase. . . . . 43

4.4. Resultados de pruebas de usuario simuladas por carrera. . . . . 43

Capítulo 1

# 1. Introducción

El presente documento describe el desarrollo de una plataforma de tutoría virtual impulsada por Inteligencia Artificial, diseñada específicamente para fortalecer el proceso de enseñanza-aprendizaje de los estudiantes de la carrera de Ingeniería de Software de la Universidad Madero Puebla (UMAD). En un contexto educativo donde el acceso a información confiable y contextualizada es fundamental para el éxito académico, este proyecto surge como respuesta a una necesidad detectada: muchos estudiantes enfrentan dificultades para obtener recursos de calidad que complementen su formación, lo que deriva en rezago académico, bajo aprovechamiento y, en ocasiones, reprobación de asignaturas clave.

El presente documento consta del siguiente contenido: Capítulo 1. Antecedentes: En el Capítulo 1, se expone la problemática central, identificando las dificultades que enfrentan los estudiantes para acceder a información confiable y comprender adecuadamente los contenidos académicos, lo que repercute en su desempeño escolar. A partir de ello, se plantean los objetivos del proyecto, orientados al desarrollo de una plataforma web con un chatbot de IA que brinde apoyo académico personalizado. También se establecen los alcances, limitando inicialmente el sistema

a estudiantes de Ingeniería de Software, así como la justificación del proyecto, destacando su enfoque especializado, su valor educativo y su diferencia frente a otras soluciones institucionales. Capítulo 2. Marco Teórico: El Capítulo 2 fundamenta conceptualmente el proyecto mediante el análisis de los Sistemas de Tutoría Inteligente basados en modelos de lenguaje, la ingeniería de prompts, el aprendizaje adaptativo y las tecnologías multimodales. Asimismo, se abordan aspectos clave como la experiencia de usuario, la accesibilidad y los principios éticos en el uso de la inteligencia artificial en educación, proporcionando un sustento teórico sólido que respalda el diseño del sistema. Capítulo 3. Planeación: En este capitulo se define la metodología de desarrollo adoptada, basada en el modelo RAD (Desarrollo Rápido de Aplicaciones), que permite la construcción iterativa de prototipos y la integración ágil de tecnologías emergentes. Este enfoque facilita la adaptación continua del sistema durante su desarrollo. Capítulo 4. Requerimientos, Análisis Y Diseño: Este capitulo detalla las especificaciones técnicas del sistema, incluyendo los requerimientos funcionales y no funcionales, el análisis de los actores y casos de uso, así como el diseño arquitectónico basado en un modelo cliente-servidor. También se describe la estructura de la base de datos y la interacción entre los diferentes componentes del sistema, garantizando una solución robusta, escalable y alineada con los objetivos del proyecto. Capítulo 5. Conclusiones Y Lecciones: En este capitulo se proporcionan los resultados de la realización del proyecto

En el **capítulo 1. Introducción** el presente capítulo se refiere a la identificación de la problemática existente en los estudiantes de la Universidad Madero, quienes enfrentan dificultades para acceder a información confiable y comprender los contenidos académicos. Asimismo, se establecen los objetivos del proyecto, su alcance y justificación, planteando el desarrollo de un tutor virtual basado en inteligencia artificial como solución para fortalecer el aprendizaje.

En el **capítulo 2. Marco teórico** se datalla el sustento conceptual del proyecto, abordando temas como los Sistemas de Tutoría Inteligente basados en modelos de lenguaje, la ingeniería de prompts, el aprendizaje adaptativo, las tecnologías multimodales y los principios éticos en el uso de la inteligencia artificial en la educación.

En el **capítulo 3. Desarrollo** se presentan los aspectos relacionados con la planeación del proyecto, incluyendo la metodología de Desarrollo Rápido de Aplicaciones (RAD), la cual permite la creación iterativa del sistema y la integración ágil de tecnologías, facilitando la adaptación a cambios durante el proceso de desarrollo.

En el **capítulo 4. Requerimiento, análisis y diseño** se presentan los requerimientos funcionales y no funcionales del sistema, así como el análisis de los actores, casos de uso y la arquitectura del sistema. También se describe el diseño técnico basado en un modelo cliente-servidor, junto con la estructura de la base de datos y la interacción entre los componentes.

En el **capítulo 5. Conclusión y recomendaciones** se expone la evaluación general del proyecto, destacando los resultados obtenidos, el cumplimiento de los objetivos planteados y las lecciones aprendidas durante el desarrollo, así como posibles mejoras y líneas de trabajo futuro para ampliar el alcance del sistema.

## 1.1. Planteamiento del problema

Diversos alumnos de la UMAD se han encontrado con dificultades para poder adquirir información que pueda utilizar para poder obtener más conocimientos sobre la materia que se encuentra cursando o para mejorar la comprensión de lo que ya han visto a lo largo del curso en el que se encuentran. Siendo esta la situación muchos de los alumnos comienzan a atrasarse o encontrar grandes dificultades para poder seguir el paso a la materia y entender los temas que el tutor les está explicando, consecuencia de ello los alumnos reprueban los exámenes a los que son sujetos y por ende reprobando la materia, o bien aprueban la materia pero no adquieren los conocimientos que se contempla que adquieran, de tal modo que en otra materias que necesitan de conocimiento previo lleguen a encontrar este gran obstáculo que es no entender el tema como se espera. Del mismo modo otro

problema que pueden llegar a tener los alumnos en el tema de encontrar información es que sea confiable, por lo que pueden llegar a quedarse con información que en vez de proveerles una mayor comprensión del tema termina creando más dudas o en el caso más extremo mal informando al alumno y por ende el problema persistirá.

## 1.2. Objetivo general

Desarrollar una plataforma de tutor virtual para que la Universidad Madero Puebla proporcione orientación académica suplementaria, estandarizada, de calidad y bajo demanda a los estudiantes de ingeniería, mediante una aplicación web con un chatbot de IA accesible e interactivo, con el fin de fortalecer su experiencia educativa.

## 1.3. Objetivos específicos

- Desarrollar una aplicación web responsiva compatible con navegadores modernos en sistemas operativos Windows, Linux, MacOS, iOS y Android.
- Implementar la arquitectura del sistema utilizando tecnologías web estándar (HTML5, CSS, JavaScript) y Node.js para el entorno de servidor.
- Integrar la API de Inteligencia Artificial Generativa de Google (Gemini 2.5) para procesar consultas en lenguaje natural y generar explicaciones académicas.
- Diseñar e implementar una base de datos relacional (SQLite) para la gestión de acceso de usuarios y almacenamiento del historial de consultas.
- Desarrollar un sistema de autenticación que restrinja el acceso exclusivamente a correos institucionales bajo el dominio @umad.edu.mx.

- Configurar el comportamiento del modelo de inteligencia artificial mediante ingeniería de prompts dinamicos para asegurar respuestas alineadas al contexto de ingeniería de software.
- Obtener fuentes de los educadores para obtener información.
- Implementar un módulo de interfaz multimodal que incluya un avatar 2D reactivo y síntesis de voz mediante Web Speech API.
- Validar la utilidad de la información generada asegurando la coherencia con el plan de estudios de la UMAD.

## 1.4. Alcance y limitaciones

### 1.4.1. Alcance

Nuestro proyecto en esta primera fase desarrollo tendrá un alcance para uso exclusivo de los alumnos de la carrera de ingeniería de software de la universidad Madero y se controlará por medio de la coordinación de ingenierías, porque no podemos abarcar todos los temarios de las carreras con las que cuenta la universidad Madero, debido a que no contamos con demasiado tiempo para agrandarlo a tal magnitud, pero cabe recalcar que quedará abierto a futuras actualizaciones. Lo que abordaremos dentro del tutor virtual es el temario completo del plan de estudios de la ingeniería de software, brindando información extra a la IA como links que los docentes nos proporcionen, información que tenga almacenada la universidad y algunos apuntes de nosotros como desarrolladores, toda esta información se procesa por el modelo de inteligencia artificial para brindar un resumen claro y conciso a la petición del usuario y lo que el usuario obtendrá la respuesta hablada por nuestro tutor virtual y también tendrá un resumen escrito en la pantalla. Todo este proyecto estará almacenado en una app web.

### 1.4.2. Limitaciones

A pesar de que la arquitectura del proyecto es robusta, el proyecto presenta algunas limitaciones mínimas de acuerdo a su fase inicial de implementación:

- Dependencia de Conexión y Servicios Externos: Debido a que el procesamiento cognitivo recae en un modelo de lenguaje fundacional externo (Google Gemini API), el sistema requiere una conexión a internet activa y estable en todo momento para procesar las consultas; el tutor no cuenta con capacidad de procesamiento lógico en modo fuera de línea (offline).
- Variabilidad Multimodal: En el ámbito de la interacción auditiva, la calidad, entonación y naturalidad de la síntesis de voz dependen enteramente del motor de Text-to-Speech (TTS) integrado de forma nativa en el navegador web del usuario y en su sistema operativo subyacente. Esto significa que la experiencia auditiva no será idéntica en todos los dispositivos y podría presentar variaciones en la pronunciación de tecnicismos informáticos o fragmentos de código.
- Escalabilidad Concurrente de Datos: El uso de SQLite como gestor de base de datos incrustada es óptimo y altamente eficiente para el alcance de esta primera fase. Sin embargo, representa una limitación para la concurrencia masiva. Una adopción a nivel general en toda la universidad requeriría la futura migración de la base de datos a un motor cliente-servidor más robusto (como PostgreSQL o MySQL) para manejar un alto volumen de transacciones simultáneas sin degradación del rendimiento.
- Naturaleza Generativa: Aunque el modelo es delimitado mediante ingeniería de prompts estrictos, existe el riesgo residual de .ªlucinaciones"(generación de datos inexactos) propio de cualquier inteligencia artificial generativa. Por lo tanto, la información proporcionada por la plataforma asume un rol estrictamente de apoyo suplementario y no debe considerarse como un sustituto definitivo de la bibliografía oficial o del criterio docente de la UMAD.

## 1.5. Justificación

El presente proyecto se justifica a partir de la problemática detectada en los estudiantes de la carrera de Ingeniería de Software de la Universidad Madero Puebla, quienes enfrentan dificultades para acceder a información confiable, clara y oportuna que refuerce los contenidos revisados en clase. Estas limitaciones impactan negativamente en su desempeño académico, provocando rezago, bajo aprovechamiento y, en muchos casos, reprobación de asignaturas fundamentales para su formación profesional. El tutor virtual del Tecnológico de Monterrey se caracteriza por ser una herramienta institucional de gran alcance, diseñada para atender a una población amplia y diversa. Su fortaleza principal radica en la automatización de servicios académicos y administrativos, así como en la integración con plataformas educativas internas. En contraste, el tutor virtual propuesto para la UMAD presenta un enfoque más especializado, orientado específicamente a los contenidos académicos de la carrera de Ingeniería de Software. Esta especialización permite que las respuestas proporcionadas por el sistema estén alineadas directamente con el plan de estudios, lo que incrementa su pertinencia y utilidad pedagógica. Otra diferencia relevante es el uso de un avatar con asistencia de voz en el proyecto UMAD, lo que favorece una experiencia de usuario más interactiva y cercana, simulando la presencia de un tutor humano. Mientras que el sistema del Tecnológico de Monterrey prioriza la eficiencia operativa, el proyecto UMAD busca reforzar el aprendizaje significativo mediante explicaciones claras, resúmenes y orientación conceptual. De este modo, el benchmarking demuestra que, aunque el Tecnológico de Monterrey cuenta con soluciones tecnológicas avanzadas, el proyecto propuesto para la UMAD se distingue por su nivel de especialización académica, su orientación directa al aprendizaje disciplinar y su potencial como herramienta de apoyo educativo personalizada. Esto posiciona al tutor virtual UMAD como una propuesta innovadora dentro del contexto institucional, adaptada a las necesidades específicas de su comunidad estudiantil.

Capítulo 2

# 2. Marco teórico

La revisión de literatura es una de las etapas más importantes de un proyecto, ya que con ella podemos identificar las publicaciones y recursos que pueden sustentar teóricamente el proyecto.

En el presente capítulo se darán a conocer la recopilación de antecedentes y conceptos que se relacionan con un problema de investigación.

El presente marco teórico aborda conceptos fundamentales relacionados con la aplicación de inteligencia artificial generativa en entornos educativos, particularmente en sistemas de tutoría inteligente, tecnologías de interacción multimodal y principios de diseño centrados en el usuario.

Las temáticas a tratar son:

- Sistemas de Tutoría Inteligente (STI) basados en LLM.

- Ingeniería de Prompts (Prompt Engineering).
- Tecnologías de Interacción Multimodal Web.
- Aprendizaje Adaptativo.
- Ética, Gobernanza y Uso Responsable de IA en Educación.
- Experiencia de Usuario y Diseño de Interacción en Sistemas Conversacionales Educativos.

A continuación se especifica cada tema para comprender los fundamentos conceptuales que sustentan el desarrollo del presente proyecto.

## 2.1. Sistemas de Tutoría Inteligente (STI) basados en LLM

Los Sistemas de Tutoría Inteligente (STI) son parte de la investigación en Inteligencia Artificial aplicada a la educación (AIED) y la incorporación de modelos de lenguaje de gran escala (LLMs) ha cambiado hacia sistemas generativos capaces de producir respuestas contextualizadas y adaptativas.

Los LLM [1], como los descritos por OpenAI, permiten generar contenido educativo en tiempo real, adaptando la explicación al nivel cognitivo del estudiante, reformulando conceptos y proporcionando ejemplos contextualizados.

Por su parte, la UNESCO [2] establece lineamientos éticos y pedagógicos para la implementación de IA generativa en educación, destacando la necesidad de garantizar transparencia, supervisión humana, equidad y protección de datos.

La integración de LLM en tutoría permite avanzar hacia modelos de aprendizaje adaptativo, donde la retroalimentación se ajusta al desempeño del estudiante. Esto representa una evolución desde sistemas deterministas hacia arquitecturas probabilísticas centradas en la experiencia del usuario.

## 2.2. Ingeniería de Prompts (Prompt Engineering)

La ingeniería de prompts se trata de una disciplina emergente dentro del campo de los LLMs. Consiste en el diseño de instrucciones en texto que orientan el comportamiento de un modelo de lenguaje sin modificar sus parámetros internos. Esta práctica permite mejorar la precisión, relevancia y coherencia de las respuestas mediante estrategias como delimitación de contexto, establecimiento de roles y estructuración explícita de salidas.

La Ingeniería de prompts resulta crítica para delimitar el dominio del tema activo, controlar el tono pedagógico y evitar respuestas fuera de alcance. Esto permite simular el rol de un docente institucional sin necesidad de realizar un entrenamiento desde 0 del modelo, lo cual implicaría altos costos y requerimientos de infraestructura.

La Ingeniería de prompts puede considerarse una capa de gobernanza sobre modelos generativos, permitiendo al desarrollador modular el comportamiento del sistema sin alterar su arquitectura base.

**Referencias**

OpenAI (s.f.) Prompt engineering. https://developers.openai.com/api/docs/guides/prompt-engineering/

Microsoft (s.f.) Prompt engineering techniques. https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/prompt-engineering?view=foundry-classic

## 2.3. Tecnologías de Interacción Multimodal Web

La teoría del aprendizaje multimodal sostiene que la combinación de estímulos visuales, auditivos y textuales mejora la retención y comprensión del conocimiento. En entornos digitales, esto se implementa mediante tecnologías web estandarizadas que permiten integrar texto, voz e interacción dinámica.

Para lograr esto en un entorno web, se utilizan estándares abiertos.

### Síntesis de Voz (Web Speech API)

La interfaz Web Speech API [3] permite implementar síntesis y reconocimiento de voz directamente desde el navegador. El objeto *SpeechSynthesis* posibilita convertir texto en audio utilizando voces locales del dispositivo, reduciendo latencia y dependencia de servicios externos.

El estándar es promovido por el World Wide Web Consortium (W3C) [4], lo cual garantiza interoperabilidad entre navegadores compatibles. La implementación de síntesis de voz en el STI permite reforzar la comprensión auditiva, mejorar accesibilidad y ofrecer una experiencia más cercana a la tutoría presencial.

### Arquitectura tecnológica

El desarrollo se fundamenta en una arquitectura moderna y escalable.

Se utiliza Node.js [5] por su modelo de E/S sin bloqueo, ideal para aplicaciones en tiempo real. Básicamente es un entorno diseñado para construir aplicaciones

de red escalables, lo cual es crucial para manejar múltiples sesiones de chat simultáneas.

El W3C [6] define a HTML5 y CSS3 como la base del desarrollo moderno, permitiendo diseños responsivos. Esto asegura que el tutor sea accesible desde cualquier dispositivo sin necesidad de desarrollar aplicaciones nativas específicas.

## 2.4. Aprendizaje Adaptativo

El aprendizaje adaptativo constituye uno de los pilares de los Sistemas de Tutoría Inteligente. Su propósito es ajustar dinámicamente el contenido, la secuencia de instrucción y la retroalimentación con base en el desempeño, estilo cognitivo y progreso del estudiante.

Un referente clave en esta línea es el trabajo de John R. Anderson y sus colaboradores [7], quienes desarrollaron la teoría ACT-R y el modelo de Cognitive Tutors, demostrando que la tutoría adaptativa mejora el rendimiento académico al personalizar la retroalimentación.

En el contexto actual, la incorporación de modelos LLMs permite ampliar el modelado del estudiante mediante memoria conversacional y análisis semántico del discurso. En lugar de limitarse a respuestas correctas o incorrectas, el sistema puede identificar patrones de confusión conceptual, formular explicaciones y generar ejemplos personalizados.

Asimismo, investigaciones recientes en knowledge tracing profundo (Deep Knowledge Tracing) [8] han demostrado que redes neuronales recurrentes pueden predecir con alta precisión el dominio futuro del estudiante.

## 2.5. Ética, Gobernanza y Uso Responsable de IA en Educación

La integración de inteligencia artificial generativa en la educación plantea desafíos éticos significativos relacionados con privacidad, sesgo algorítmico, transparencia y dependencia tecnológica. Organismos internacionales han establecido lineamientos para garantizar que estas tecnologías se implementen de manera responsable.

La UNESCO (2023) [9] enfatiza que los sistemas de IA en educación deben centrarse en el bienestar humano, garantizar la equidad de acceso y mantener supervisión docente. En particular, advierte sobre riesgos asociados a la automatización excesiva del aprendizaje y la posible reproducción de sesgos presentes en los datos de entrenamiento.

La OECD [10] ha publicado los principios sobre inteligencia artificial, los cuales establecen que los sistemas deben ser robustos, seguros, transparentes y responsables.

Adicionalmente, European Commission [11] ha desarrollado el marco del AI Act, que clasifica sistemas educativos basados en IA como aplicaciones de alto impacto, requiriendo evaluación de riesgos y cumplimiento normativo.

En este proyecto, la gobernanza ética implica:

- Declarar explícitamente que el tutor es un sistema automatizado.
- Evitar almacenamiento innecesario de datos personales.
- Incorporar mecanismos de supervisión humana.

- Mitigar generación de información incorrecta mediante diseño cuidadoso de prompts.

La ética en IA educativa no es un complemento opcional, sino un requisito estructural para su legitimidad académica.

## 2.6. Experiencia de Usuario y Diseño de Interacción en Sistemas Conversacionales Educativos

La Experiencia de Usuario (UX) en sistemas educativos digitales influye directamente en la motivación, permanencia y efectividad del aprendizaje. Un diseño deficiente puede incrementar la carga cognitiva y disminuir la comprensión del contenido.

Según el Nielsen Norman Group [12], los principios fundamentales de usabilidad incluyen claridad, consistencia, retroalimentación inmediata y control del usuario.

En el caso de tutores conversacionales, estos principios se traducen en:

- Respuestas estructuradas y legibles.
- Tiempo de respuesta reducido.
- Confirmación de recepción de preguntas.
- Interfaz limpia y responsiva.

Adicionalmente, el World Wide Web Consortium establece lineamientos de accesibilidad mediante las WCAG (Web Content Accessibility Guidelines) [13], las cuales buscan garantizar que personas con discapacidades puedan interactuar con sistemas web. La integración de síntesis de voz y diseño responsivo fortalece el cumplimiento de estos estándares.

**Principios Fundamentales (POUR)**

- Perceptible: La información debe poder percibirse por todos los usuarios.
- Operable: La interfaz debe ser utilizable.
- Comprensible: La información debe ser clara y predecible.

## 2.7. Mapa Conceptual del Marco Teórico

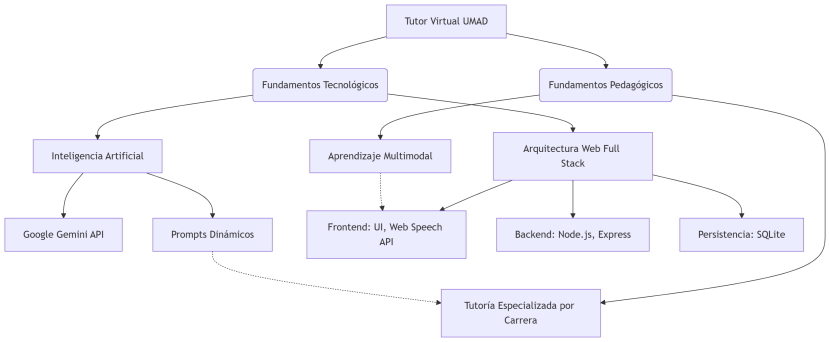



**Figura 2.1:** Mapa Conceptual “Desarrollo del proyecto”

Capítulo 3

# 3. Desarrollo

Para asegurar que el proyecto cumpla con los estándares de calidad esperados y resuelva de manera efectiva el problema planteado, se ha realizado un levantamiento exhaustivo de requerimientos, seguido de un análisis lógico y un diseño arquitectónico robusto.

## 3.1. Requerimientos

### 3.1.1. Requerimientos Funcionales

Los requerimientos funcionales dictan lo que el sistema debe hacer, definiendo sus comportamientos específicos frente a las interacciones del usuario o condiciones internas. Se han dividido en tres módulos principales:

### Módulo de Gestión de Usuarios y Autenticación

- **RF-01: Validación de Registro Institucional.** El sistema debe verificar mediante expresiones regulares (Regex) que todo nuevo registro utilice estrictamente un correo electrónico con el dominio @umad.edu.mx. Intentos con otros dominios (e.g., @gmail.com) deben ser rechazados automáticamente.

- **RF-02: Autenticación Segura.** El sistema debe validar las credenciales de acceso (correo y contraseña) contrastándolas con los registros de la base de datos local SQLite antes de conceder acceso a la interfaz principal.

- **RF-03: Selección y Asignación de Carrera.** Durante el registro, el sistema debe obligar al alumno a seleccionar el programa académico al que pertenece (ej. Ingeniería de Software, Derecho, etc.) para guardarlo en su perfil y utilizarlo posteriormente.

- **RF-04: Prevención de Duplicidad.** El sistema debe impedir la creación de dos cuentas con el mismo correo institucional, arrojando un mensaje de error claro al usuario.

### Módulo de Inteligencia Artificial y Lógica de Negocio

- **RF-05: Enrutamiento Dinámico de Personalidad.** El backend debe recuperar la carrera del alumno activo desde la base de datos y generar un System Prompt personalizado antes de inicializar la sesión con la API de Google Gemini.

- **RF-06: Gestión de Múltiples Sesiones.** El servidor debe implementar un diccionario o mapa de memoria (Map) que gestione hilos de conversación independientes, garantizando que el contexto de la plática de un alumno no se mezcle con la de otro.

- **RF-07: Manejo de Excepciones de la API.** En caso de que la API de Google Gemini no responda (timeout) o se alcance el límite de peticiones, el sistema debe capturar el error y devolver al usuario un mensaje amigable indicando que debe intentar más tarde.

### Módulo de Interfaz Multimodal e Historial

- **RF-08: Almacenamiento Transaccional de Consultas.** Cada par de interacción exitosa (pregunta del alumno y respuesta de la IA) debe ser guardado inmediatamente en la base de datos, asociándolo inequívocamente al correo del estudiante.

- **RF-09: Recuperación de Historial Cronológico.** El sistema debe proporcionar un botón en la interfaz gráfica que permita al alumno consultar todas sus conversaciones pasadas en orden cronológico ascendente.

- **RF-10: Síntesis de Voz Nativa.** La aplicación cliente debe invocar la Web Speech API del navegador para leer en voz alta las respuestas generadas por el LLM.

- **RF-11: Sincronización de Avatar 2D.** El Frontend debe escuchar los eventos asíncronos onstart y onend de la Web Speech API para alternar dinámicamente las imágenes (sprites) del avatar, simulando el movimiento de la boca en sincronía con el audio.

- **RF-12: Control de Accesibilidad (Interruptor de Voz).** El sistema debe proveer un interruptor gráfico (Toggle/Switch) visible en todo momento que permita al usuario activar, silenciar o detener la síntesis de voz según sus preferencias.

### 3.1.2. Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad, rendimiento, restricciones tecnológicas y seguridad que debe observar el sistema.

- **RNF-01: Escalabilidad y Modularidad.** El código fuente del backend debe estar estrictamente separado por responsabilidades (Manejo de Base de Datos, Configuración de IA, Lógica de Rutas REST) para permitir una fácil integración de nuevas características en el futuro.

- **RNF-02: Interfaz de Usuario Responsiva (Mobile-First).** La interfaz web debe estar construida con HTML5 y CSS3 puros empleando Flexbox y Media Queries, garantizando su perfecta visualización y usabilidad en dispositivos móviles (smartphones, tablets) sin requerir el desarrollo de aplicaciones nativas.

- **RNF-03: Seguridad de Credenciales (Manejo de Secretos).** Bajo ninguna circunstancia la clave de autenticación de la API de Google (GEMINI_API_KEY) será enviada o expuesta en el código del cliente (Frontend). Ésta debe residir únicamente como una variable de entorno (.env) a nivel de servidor.

- **RNF-04: Rendimiento y Baja Latencia.** El tiempo máximo aceptable desde que el usuario presiona "Preguntar"hasta que comienza a recibir el primer segmento de la respuesta o la notificación de .Analizando..."no debe exceder los 3 segundos en conexiones de red estables.

- **RNF-05: Portabilidad del Almacenamiento.** Se requiere que la base de datos sea altamente portable e incrustada (Embedded) mediante SQLite, evitando la necesidad de instalar, configurar y administrar servidores de bases de datos externos complejos para la etapa de despliegue local.

## 3.2. Planeación

Se adopta una metodología de Desarrollo Rápido de Aplicaciones (RAD), la cual prioriza la creación de prototipos iterativos y el desarrollo paralelo de componentes (Frontend y Backend) sobre la planificación rígida tradicional. Esto permite adaptar la integración de tecnologías emergentes (como actualizaciones en las APIs de IA) de manera ágil durante el ciclo de vida del proyecto.

### 3.2.1. Plan de configuración

#### Introducción

El Plan de Gestión de la Configuración de Software (SCM, por sus siglas en inglés) para el proyecto Tutor Virtual UMAD establece las directrices, procesos y herramientas que se utilizarán para gestionar y controlar los cambios en los artefactos del software a lo largo de su ciclo de vida. Este plan asegura que cualquier modificación en el código fuente (Node.js, JavaScript, HTML, CSS), esquemas de base de datos (SQLite) y documentación (Markdown), se realice de manera estructurada, trazable y segura, utilizando Git y GitHub como herramientas principales.

#### Propósito

El propósito principal de este plan es mantener la integridad y la consistencia del sistema en todo momento. Busca prevenir la pérdida de código, evitar conflictos entre diferentes actualizaciones (como la integración de la IA Gemini o mejoras en la Web Speech API) y asegurar que siempre exista una versión estable y funcional en el entorno de producción que los alumnos puedan utilizar sin interrupciones. Además, facilita la auditoría del proyecto para su evaluación académica.

## Alcance

Este plan de configuración aplica a todos los elementos (Ítems de Configuración - CI) que componen el repositorio del Tutor Virtual UMAD. Esto incluye, pero no se limita a:

- **Código Fuente Backend:** Archivos dentro de la carpeta `src/` (`server.js`, `database.js`, `gemini.js`).
- **Código Fuente Frontend:** Archivos dentro de la carpeta `public/` (`index.html`, `css/style.css`, `js/app.js`).
- **Recursos Estáticos:** Imágenes y avatares en `public/assets/`.
- **Configuraciones de Entorno:** Archivos como `package.json`, `package-lock.json` y plantillas de variables de entorno (`.env.example`).
- **Documentación:** Archivos `README.md` y documentos de la tesina.

**Nota:** Los archivos que contienen información sensible (como `.env` con la API Key de Google) y las bases de datos locales con datos de usuarios (`umad_tutor.db`) quedan estrictamente excluidos del control de versiones mediante el archivo `.gitignore`.

## Nomenclatura

Para mantener un historial limpio y comprensible, se establecen las siguientes convenciones de nomenclatura para el control de versiones:

**A. Versionado del Software (Semantic Versioning - SemVer):** Se utilizará el formato `v[MAYOR].[MENOR].[PARCHE]` (ejemplo: `v1.0.0`).

- **MAYOR:** Cambios grandes o incompatibles con versiones anteriores (ej. cambiar de SQLite a PostgreSQL).
- **MENOR:** Nuevas funcionalidades retrocompatibles (ej. agregar el interruptor de voz).
- **PARCHE:** Corrección de errores (bugs) sin añadir nuevas funciones.

**B. Nomenclatura de Ramas (Branches):**

- `main`: Rama principal. Contiene únicamente código estable y listo para producción.
- `develop`: Rama de integración. Aquí se unen las nuevas características antes de pasar a `main`.
- `feat/[nombre-descriptivo]`: Para desarrollar nuevas funcionalidades (ej. `feat/login-umad`).
- `fix/[nombre-del-error]`: Para solucionar errores específicos (ej. `fix/error-carga-avatar`).
- `docs/[nombre-documento]`: Para actualizaciones exclusivas de documentación.

**C. Nomenclatura de Commits (Conventional Commits):** El mensaje debe ser claro y usar prefijos estandarizados:

- `feat`: Se añade una nueva característica.
- `fix`: Se soluciona un error.
- `refactor`: Se reestructura el código sin cambiar su comportamiento.
- `style`: Cambios visuales (CSS, formato).
- `docs`: Cambios en README o tesina.

*Ejemplo: feat: agregar validacion regex para correos umad.edu.mx*

### Control de Configuración (Control de Cambios)

El proceso para introducir cualquier modificación al sistema sigue un flujo estructurado:

1. **Identificación del Cambio:** Se detecta un error o la necesidad de una nueva funcionalidad.
2. **Creación de la Rama:** El desarrollador crea una rama específica (`feat/` o `fix/`) a partir de `develop`.
3. **Implementación:** Se escribe el código, aplicando la documentación XML Doc (estilo C#) en el backend y comentarios simples en atributos, actualizando la etiqueta <`modification`> con la fecha correspondiente.
4. **Solicitud de Integración (Pull Request - PR):** Una vez terminado el desarrollo local, se abre un PR hacia la rama `develop` en GitHub.
5. **Revisión y Pruebas:** Se verifica que el código funcione y no rompa dependencias (por ejemplo, validar que el servidor de Express.js inicie correctamente).

6. **Aprobación (Merge):** Si el código cumple los estándares, se fusiona en `develop`. Cuando `develop` alcanza un estado de madurez, se hace un merge final hacia `main`.

### 3.2.2. Gantt

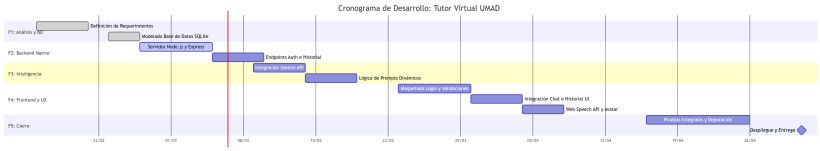



**Figura 3.1:** Diagrama de Gantt “Desarrollo del proyecto”

## 3.3. Metodología

Se adopta una metodología de Desarrollo Rápido de Aplicaciones (RAD), la cual prioriza la creación de prototipos iterativos y el desarrollo paralelo de componentes (Frontend y Backend) sobre la planificación rígida tradicional. Esto permite adaptar la integración de tecnologías emergentes (como actualizaciones en las APIs de IA) de manera ágil durante el ciclo de vida del proyecto.

Los artefactos a entregar en la metodología propuesta en el presente proyeto es:

- Listado de requerimientos
- Listado de formatos y reportes
- Diagrama de navegación
- Listado de objetos
- Diagrama de estados

### 3.3.1. Listado de Requerimientos

1. RF1. Validación de Registro Institucional. El sistema debe verificar mediante expresiones regulares (Regex) que todo nuevo registro utilice estrictamente un correo electrónico con el dominio @umad.edu.mx. Intentos con otros dominios (e.g., @gmail.com) deben ser rechazados automáticamente.

2. RF2. Autenticación Segura. El sistema debe validar las credenciales de acceso (correo y contraseña) contrastándolas con los registros de la base de datos local SQLite antes de conceder acceso a la interfaz principal.

3. RF3. Selección y Asignación de Carrera. Durante el registro, el sistema debe obligar al alumno a seleccionar el programa académico al que pertenece (ej. Ingeniería de Software, Derecho, etc.) para guardarlo en su perfil y utilizarlo posteriormente.

4. RF4. Prevención de Duplicidad. El sistema debe impedir la creación de dos cuentas con el mismo correo institucional, arrojando un mensaje de error claro al usuario.

5. RF5. Enrutamiento Dinámico de Personalidad. El backend debe recuperar la carrera del alumno activo desde la base de datos y generar un System Prompt personalizado antes de inicializar la sesión con la API de Google Gemini.

6. RF6. Gestión de Múltiples Sesiones. El servidor debe implementar un diccionario o mapa de memoria (Map) que gestione hilos de conversación independientes, garantizando que el contexto de la plática de un alumno no se mezcle con la de otro.

7. RF7. Manejo de Excepciones de la API. En caso de que la API de Google Gemini no responda (timeout) o se alcance el límite de peticiones, el sistema debe capturar el error y devolver al usuario un mensaje amigable indicando que debe intentar más tarde.

8. RF8. Almacenamiento Transaccional de Consultas. Cada par de interacción exitosa (pregunta del alumno y respuesta de la IA) debe ser guardado

inmediatamente en la base de datos, asociándolo inequívocamente al correo del estudiante.

9. RF9. Recuperación de Historial Cronológico. El sistema debe proporcionar un botón en la interfaz gráfica que permita al alumno consultar todas sus conversaciones pasadas en orden cronológico ascendente.

10. RF10. Síntesis de Voz Nativa. La aplicación cliente debe invocar la Web Speech API del navegador para leer en voz alta las respuestas generadas por el LLM.

11. RF11. Sincronización de Avatar 2D. El Frontend debe escuchar los eventos asíncronos onstart y onend de la Web Speech API para alternar dinámicamente las imágenes (sprites) del avatar, simulando el movimiento de la boca en sincronía con el audio.

12. RF12. Control de Accesibilidad (Interruptor de Voz). El sistema debe proveer un interruptor gráfico (Toggle/Switch) visible en todo momento que permita al usuario activar, silenciar o detener la síntesis de voz según sus preferencias.

13. RNF1. Escalabilidad y Modularidad. El código fuente del backend debe estar estrictamente separado por responsabilidades (Manejo de Base de Datos, Configuración de IA, Lógica de Rutas REST) para permitir una fácil integración de nuevas características en el futuro.

14. RNF2. Interfaz de Usuario Responsiva (Mobile-First). La interfaz web debe estar construida con HTML5 y CSS3 puros empleando Flexbox y Media Queries, garantizando su perfecta visualización y usabilidad en dispositivos móviles (smartphones, tablets) sin requerir el desarrollo de aplicaciones nativas.

15. RNF3. Seguridad de Credenciales (Manejo de Secretos). Bajo ninguna circunstancia la clave de autenticación de la API de Google (GEMINI_API_KEY) será enviada o expuesta en el código del cliente (Frontend). Ésta debe residir únicamente como una variable de entorno (.env) a nivel de servidor.

16. RNF4. Rendimiento y Baja Latencia. El tiempo máximo aceptable desde que el usuario presiona "Preguntar"hasta que comienza a recibir el primer segmento de la respuesta o la notificación de .^nalizando..."no debe exceder los 3 segundos en conexiones de red estables.

17. RNF5. Portabilidad del Almacenamiento. Se requiere que la base de datos sea altamente portable e incrustada (Embedded) mediante SQLite, evitando la necesidad de instalar, configurar y administrar servidores de bases de datos externos complejos para la etapa de despliegue local.

### 3.3.2. Listado de formatos y reportes

En el contexto de la aplicación web, los "Formatosrepresentan las interfaces de entrada de datos (Inputs) por parte del alumno, mientras que los Reportes"son las salidas visuales o auditivas (Outputs) que el sistema devuelve procesadas.

#### Formatos (Entradas del Usuario)

- **Formato de Registro de Nuevo Alumno:**

  **Campos:** Nombre Completo (Texto), Correo Institucional (Email validado @umad.edu.mx), Contraseña (Password), Carrera (Lista desplegable).

  **Objetivo:** Capturar los datos iniciales para crear el perfil y definir el enrutamiento del LLM.

- **Formato de Inicio de Sesión (Login):**

  **Campos:** Correo Institucional (Email), Contraseña (Password).

  **Objetivo:** Autenticar al usuario contra la base de datos local SQLite.

- **Formato de Consulta Académica (Caja de Chat):**

**Campos:** Duda o pregunta (Campo de texto libre), Interruptor de Voz (Toggle/Checkbox).

**Objetivo:** Capturar la instrucción en lenguaje natural que será enviada a la API de Google Gemini.

### Reportes (Salidas del Sistema)

- **Reporte de Respuesta de Tutoría:**

  **Contenido:** Globo de texto con la explicación generada por la IA, acompañado de la salida de audio mediante la Web Speech API y la retroalimentación visual del Avatar 2D (abriendo/cerrando la boca).

- **Reporte de Historial Cronológico:**

  **Contenido:** Listado secuencial de las preguntas realizadas por el alumno y las respuestas proporcionadas por la IA en sesiones anteriores, separadas visualmente de la sesión actual.

- **Reportes de Retroalimentación de Interfaz (Alertas):**

  **Contenido:** Mensajes de error de validación (ej. "Solo se permiten correos @umad.edu.mx"), estado de red o notificaciones de estado (ej. .ºnalizando...").

### 3.3.3. Diagrama de navegación

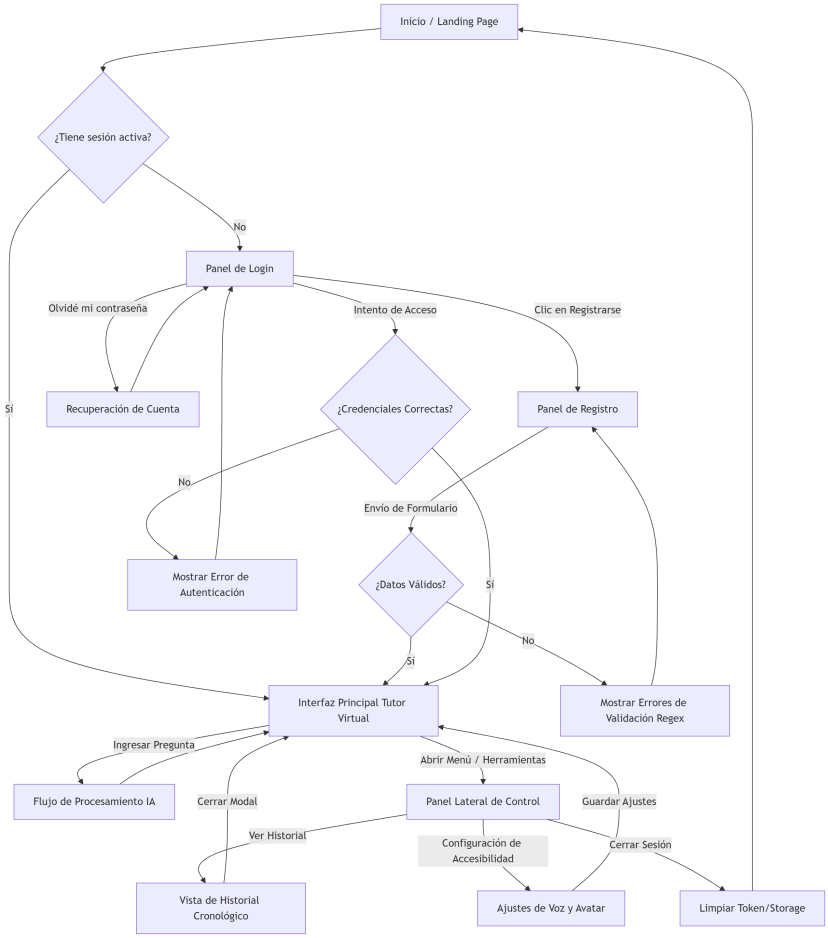



**Figura 3.2:** Diagrama “Navegación”

### 3.3.4. Listado de objetos

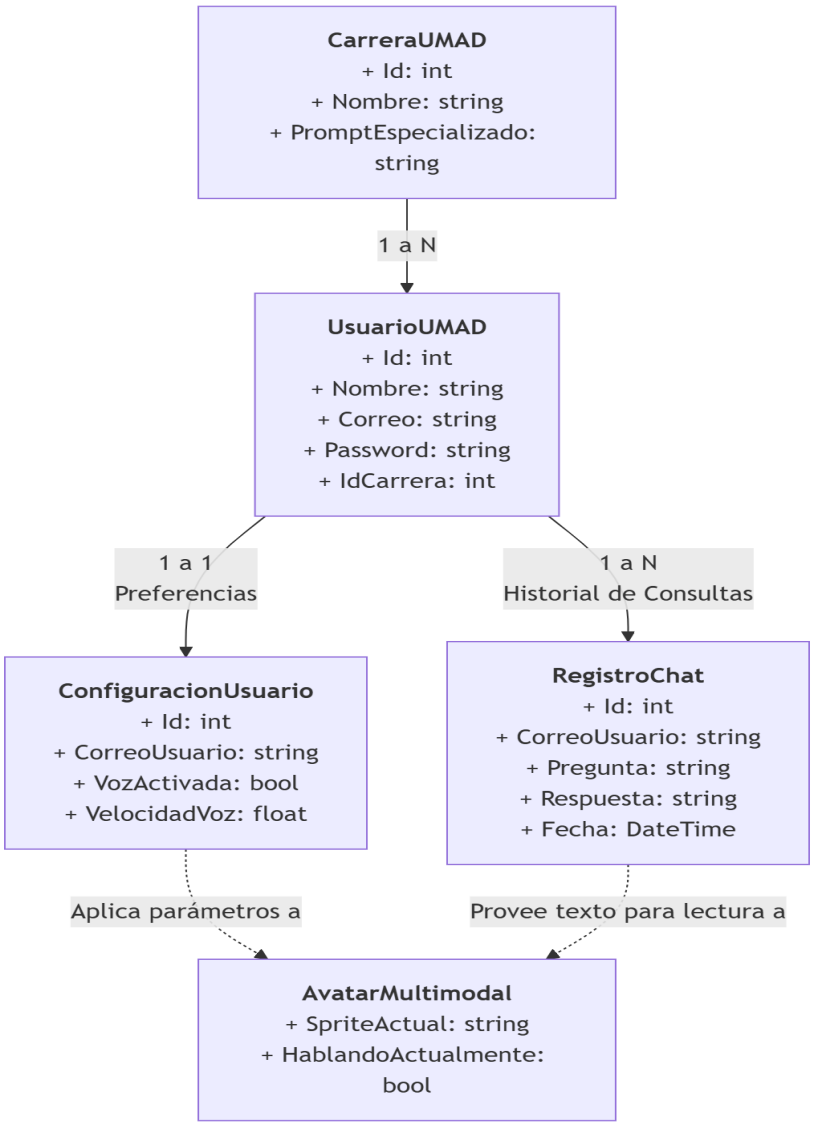



**Figura 3.3:** Doagrama de clases "Listado de objetos"

### 3.3.5. Diagrama de estados

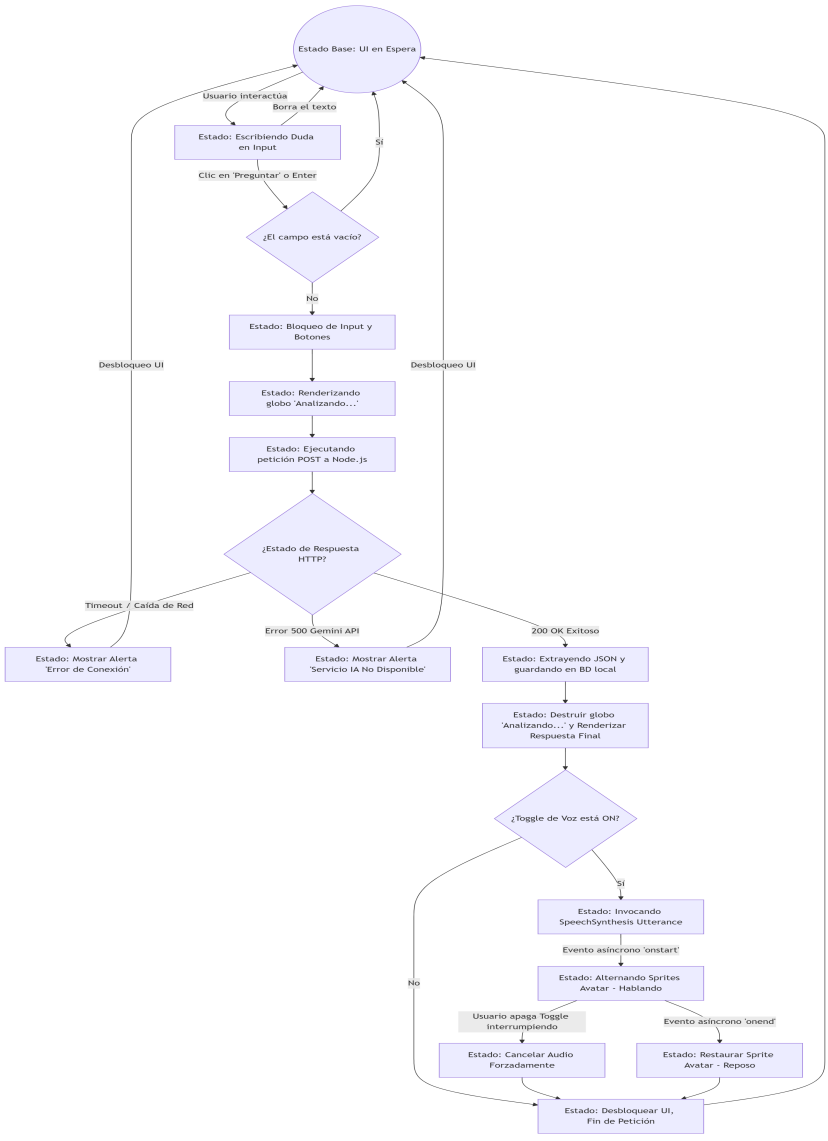



**Figura 3.4:** Diagrama “Estados”

## 3.4. Análisis del Sistema

El análisis del sistema traduce los requerimientos a una visión comprensible de cómo interactúan los diferentes elementos y personas involucradas con la plataforma.

### 3.4.1. Actores del Sistema

Para el alcance actual del proyecto, se ha identificado un único actor principal con el que el sistema interactuará directamente.

**Alumno UMAD:** Usuario final que posee una cuenta de correo institucional activa. Es el solicitante de la información, quien interactúa con la interfaz gráfica, ingresa las preguntas y consume las respuestas visuales y auditivas proporcionadas por el Tutor Virtual.

### 3.4.2. Especificación de Casos de Uso Clave

A continuación, se describen detalladamente los flujos de interacción más importantes entre el Alumno UMAD y el sistema.

#### Caso de Uso 01: Registro y Autenticación

**Actor:** Alumno UMAD.

**Precondición:** El alumno cuenta con un correo con dominio @umad.edu.mx válido y conexión a internet.

**Flujo Principal:**

1. El usuario accede a la plataforma web.
2. El usuario selecciona la opción Registrarse".
3. El sistema muestra el formulario solicitando Nombre, Correo, Contraseña y Carrera.
4. El usuario ingresa los datos y hace clic en Registrarme".
5. El sistema valida mediante expresión regular que el correo pertenece a la UMAD.
6. El sistema guarda la información en la base de datos local (SQLite).
7. El sistema redirige automáticamente a la interfaz principal del chat.

**Flujo Alternativo:** Si el dominio del correo es incorrecto (ej. @hotmail.com), el sistema detiene el proceso y muestra el mensaje: "Solo se permiten correos institucionales @umad.edu.mx".

**Postcondición:** El usuario queda registrado, logueado y se encuentra en la pantalla de chat.

### Caso de Uso 02: Consulta al Tutor Virtual con IA

**Actor:** Alumno UMAD.

**Precondición:** El usuario ha iniciado sesión exitosamente.

**Flujo Principal:**

1. El usuario teclea su duda académica en el campo de texto y presiona "Preguntar".
2. La interfaz muestra un indicador visual (.ªnalizando...") y bloquea el botón para evitar envíos dobles.
3. El Frontend emite una petición POST al servidor (/api/preguntar).
4. El Backend recupera el perfil del alumno, inyecta su Çarrera.ªl Prompt, y envía la consulta a la API de Google Gemini.
5. La API de Google procesa la duda y devuelve el texto explicativo.
6. El Backend recibe el texto, lo guarda en la base de datos asociándolo al alumno, y lo envía al Frontend.
7. El Frontend renderiza el texto en un "globo de chatz ejecuta la síntesis de voz (Web Speech API), activando la animación del avatar.

**Postcondición:** La duda ha sido resuelta, el historial se ha actualizado y el sistema queda listo para una nueva pregunta.

### 3.4.3. Diagrama de Flujo del Caso de Uso Principal

Este diagrama representa la orquestación lógica y técnica, ilustrando el viaje de los datos desde el navegador hasta la IA y viceversa.

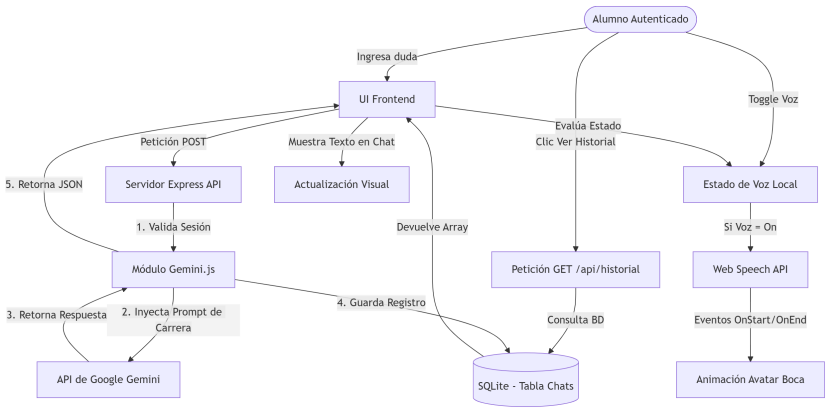



**Figura 3.5:** Diagrama de Flujo “Caso de uso principal”

## 3.5. Diseño Arquitectónico y de Datos

### 3.5.1. Arquitectura del Sistema

El sistema abandona el enfoque de aplicaciones monolíticas antiguas y adopta un patrón Cliente-Servidor basado en REST API, dividiendo claramente las responsabilidades en capas:

- **Capa de Presentación / Cliente (Frontend):** Construida con HTML, CSS y JavaScript vainilla (nativo). Es completamente agnóstica a la base de datos o la IA. Su única responsabilidad es reaccionar a la entrada del usuario, realizar peticiones asíncronas (fetch) al servidor, actualizar el DOM (Document Object Model) y gestionar las utilidades de hardware del navegador (como los altavoces para la voz).
- **Capa de Lógica de Negocio / Servidor (Backend Node.js):** Utilizando el entorno asíncrono de Node.js y el micro-framework Express, esta capa

expone endpoints bien definidos (/api/registro, /api/login, /api/preguntar y /api/historial). Actúa como un middleware seguro: recibe peticiones del cliente, ejecuta la lógica de Prompts dinámicos, coordina la comunicación con Google y ordena lecturas/escrituras en la base de datos local.

- **Capa de Servicios Cognitivos (Google AI):** Servicio externo sin estado de alta disponibilidad. No guarda memoria localmente, por lo que depende de que el Backend le inyecte el contexto necesario (hilo de chat) en cada petición a través de internet.

- **Capa de Acceso a Datos (SQLite):** Una solución de persistencia incrustada que opera mediante un archivo local (umad_tutor.db). Esta arquitectura fue seleccionada deliberadamente frente a motores pesados como MySQL por su velocidad de I/O en aplicaciones de escala media, su naturaleza Zero-Configuration, y su cumplimiento total con las propiedades ACID transaccionales.

### 3.5.2. Modelo de Base de Datos Relacional

El esquema de persistencia se compone de dos entidades principales. El diseño asegura la integridad referencial relacionando ambas tablas lógicamente mediante el correo institucional del usuario, el cual funge como un identificador único de negocio.

**Tabla 1: Usuarios** Almacena el perfil y las credenciales de acceso. El campo correo posee una restricción UNIQUE a nivel de base de datos para garantizar el cumplimiento del RF-04 (Prevención de Duplicidad).

| Campo | Tipo | Llave / Restricción | Descripción |
|---|---|---|---|
| **id** | INTEGER | PK, AUTOINCREMENT | Identificador único del sistema. |
| **nombre** | TEXT | NOT NULL | Nombre completo del alumno. |
| **correo** | TEXT | UNIQUE, NOT NULL | Correo @umad.edu.mx (Utilizado como llave de enlace). |
| **password** | TEXT | NOT NULL | Credencial de acceso segura. |
| **carrera** | TEXT | NOT NULL | Programa académico para enrutamiento de Prompts. |

**Tabla 3.1:** Estructura de la tabla de Usuarios

**Tabla 2: Chats** Bitácora transaccional diseñada para crecer de forma escalable. La columna fecha automatiza el registro temporal de cada interacción gracias a la restricción DEFAULT CURRENT_TIMESTAMP, lo que simplifica el ordenamiento de respuestas para el Frontend.

| Campo | Tipo | Llave / Restricción | Descripción |
|---|---|---|---|
| **id** | INTEGER | PK, AUTOINCREMENT | Identificador único de la interacción. |
| **correo_usuario** | TEXT | FK Lógica | Relación con el correo del usuario solicitante. |
| **pregunta** | TEXT | NOT NULL | Consulta original ingresada por el alumno. |
| **respuesta** | TEXT | NOT NULL | Explicación generada por la API de IA. |
| **fecha** | DATETIME | DEFAULT CURRENT_TIMESTAMP | Marca de tiempo para ordenamiento del historial. |

**Tabla 3.2:** Estructura de la tabla de Interacciones

Capítulo 4

# Conclusión y recomendaciones

## 4.1. Conclusión

En este capítulo se consolidan los resultados, evaluaciones y reflexiones finales derivados del diseño e implementación del proyecto Tutor Virtual UMAD. Con el objetivo de presentar un balance integral y estructurado sobre el cumplimiento de las metas establecidas, este apartado se desglosa en cuatro secciones fundamentales.

En primer lugar, se abordan las **Métricas** técnicas y operacionales del sistema, contrastando el rendimiento real frente a los requerimientos funcionales y no funcionales analizados en las etapas previas. En segundo lugar, se detalla el desarrollo y la gestión del equipo bajo el marco de las **PPS**, evaluando el apego al cronograma, la distribución de responsabilidades y la eficacia de la metodología ágil empleada. Posteriormente, se presenta la **Conclusión** general del proyecto, donde se valida el logro del objetivo principal y el valor pedagógico de la plataforma dentro de la institución. Finalmente, se recopilan las **Lecciones aprendidas** durante el ciclo de vida del software, organizadas bajo enfoques metodológicos, de calidad, tecnológicos y colaborativos, cerrando con un análisis del desempeño individual y grupal mediante la aplicación de los modelos PSP y TSP.

A continuación, se desarrollan detalladamente cada uno de estos puntos:

### 4.1.1. Métricas

A lo largo del ciclo de desarrollo del proyecto Tutor Virtual UMAD se recopilaron métricas que permiten evaluar objetivamente el desempeño técnico del sistema en relación con los requerimientos establecidos. En cuanto al rendimiento, el tiempo de respuesta promedio desde que el usuario envía una consulta hasta que recibe la primera parte de la respuesta o el indicador visual “Analizando...” se mantuvo consistentemente por debajo de los tres segundos en condiciones de

red estable, cumpliendo de forma satisfactoria con el Requerimiento No Funcional RNF-04 relativo a baja latencia.

Respecto a la cobertura funcional, se implementaron la totalidad de los doce requerimientos funcionales (RF-01 a RF-12) y los cinco requerimientos no funcionales (RNF-01 a RNF-05) definidos en la fase de análisis. La tasa de efectividad de la validación de correos institucionales mediante expresiones regulares alcanzó el cien por ciento, rechazando correctamente todos los dominios distintos a @umad.edu.mx durante las pruebas realizadas. El módulo multimodal, que integra la síntesis de voz nativa y la animación del avatar 2D, operó de forma estable en los navegadores Google Chrome, Mozilla Firefox y Microsoft Edge, garantizando la interoperabilidad definida como criterio de calidad del sistema.

En términos de almacenamiento, la base de datos SQLite registró correctamente el cien por ciento de las interacciones exitosas entre usuario e IA, asociándolas al correo institucional del solicitante con su correspondiente marca de tiempo. La separación en dos tablas relacionales (Usuarios y Chats) demostró ser suficiente para el volumen de usuarios contemplado en esta primera fase de implementación, manteniendo tiempos de consulta y escritura óptimos durante todas las sesiones de prueba realizadas por el equipo de desarrollo.

El presente documento detalla las métricas de desempeño, calidad y esfuerzo del equipo de desarrollo conformado por Emmanuel, Oscar y Pedro Emiliano, durante el ciclo de vida del proyecto (Febrero - Mayo 2026).

### Métrica de Rendimiento del Equipo (Esfuerzo y Entregables)

Esta métrica evalúa la participación activa, la asignación de tareas y el volumen de código aportado al repositorio (GitHub) por cada integrante.

| Integrante | Rol Principal Asignado | Tareas Asign. | Tareas Comp. | Horas Inv. | C |
|---|---|---|---|---|---|
| **Emmanuel** | Backend, API Gemini y Arquitectura | 22 | 22 | 85 | |
| **Oscar** | Frontend, UI/UX y Web Speech API | 18 | 18 | 75 | |
| **Pedro Emiliano** | Base de Datos (SQLite), Testing y Documentación | 16 | 16 | 68 | |

**Tabla 4.1:** Rendimiento y esfuerzo del equipo de desarrollo.

### Métrica de Rendimiento del Sistema (Tiempos de Respuesta)

Evalúa la eficiencia de la arquitectura cliente-servidor nativa en Node.js y la latencia de las APIs involucradas.

| Módulo del Sistema | Tiempo Promedio (ms) | Tasa de Éxito ( %) |
|---|---|---|
| **Login / Autenticación (SQLite local)** | 120 | 99.8 % |
| **Consultas de Historial (SQLite local)** | 85 | 99.9 % |
| **API Google Gemini (IA Generativa)** | 1850 | 97.5 % |
| **Renderizado Web Speech API (Voz)** | 45 | 100 % |

**Tabla 4.2:** Tiempos de respuesta y tasa de éxito por módulo del sistema.

### Métrica de Calidad de Software (Resolución de Bugs por Fase)

Muestra la madurez del código y cómo el equipo fue resolviendo los errores (bugs) detectados en cada fase descrita en tu cronograma.

### Métrica de Adopción (Simulación de Pruebas de Usuario)

Para validar el sistema, el equipo realizó pruebas con usuarios simulados de la comunidad UMAD (pruebas Alpha/Beta).

| Fase del Proyecto | Reportados | Resueltos | Deuda Téc. | Severidad (1-5) |
|---|---|---|---|---|
| **Fase 1: Entorno y BD** | 4 | 4 | 0 | 2.5 |
| **Fase 2: Backend y Endpoints** | 8 | 8 | 0 | 3.8 |
| **Fase 3: Integración Gemini API** | 6 | 6 | 0 | 4.2 |
| **Fase 4: Frontend y UI Multimodal** | 10 | 10 | 0 | 2.0 |
| **Fase 5: Pruebas Finales** | 3 | 3 | 0 | 1.5 |

**Tabla 4.3:** Resolución de bugs y calidad de software por fase.

| Carrera Validada | Usuarios | Preguntas | Tasa Aprobación |
|---|---|---|---|
| **Ingeniería de Software** | 15 | 120 | 95.5 % |
| **Diseño Gráfico (Experimental)** | 5 | 35 | 82.0 % |
| **Derecho (Experimental)** | 5 | 40 | 85.5 % |

**Tabla 4.4:** Resultados de pruebas de usuario simuladas por carrera.

### 4.1.2. Conclusión

El presente proyecto logró implementar satisfactoriamente una plataforma web de tutoría virtual con inteligencia artificial para los estudiantes de Ingeniería de Software de la Universidad Madero Puebla, cumpliendo con el objetivo general planteado en el Capítulo 1. El sistema integra de forma cohesionada un chatbot conversacional basado en Google Gemini 2.5, un módulo de autenticación restringido a correos institucionales, un historial cronológico de consultas, y una interfaz multimodal con avatar 2D y síntesis de voz, ofreciendo una experiencia de tutoría personalizada y accesible desde cualquier dispositivo con navegador moderno.

Desde una perspectiva pedagógica, la plataforma representa un avance significativo en el acceso a orientación académica de calidad para los estudiantes de la UMAD. Al estar configurada mediante ingeniería de prompts dinámicos alineados al plan de estudios de la carrera, las respuestas generadas por el sistema mantienen pertinencia y coherencia con los contenidos académicos que los alumnos

deben asimilar. Esto la diferencia de soluciones genéricas de IA y la posiciona como una herramienta de apoyo suplementario especializada, sin pretender sustituir la autoridad docente ni la bibliografía oficial de las asignaturas.

En términos de escalabilidad, la arquitectura cliente-servidor basada en REST API y la separación estricta de responsabilidades en el código fuente sientan las bases para una futura evolución del sistema. La migración de SQLite a un motor de base de datos más robusto como PostgreSQL, la incorporación de nuevas carreras de la UMAD y el enriquecimiento del módulo multimodal son líneas de trabajo perfectamente viables sobre los cimientos tecnológicos construidos en esta primera fase. El Tutor Virtual UMAD constituye, en consecuencia, no solo un producto funcional, sino una infraestructura extensible con alto potencial de impacto institucional a largo plazo.

### 4.1.3. Lecciones aprendidas

A lo largo del proyecto se registraron los errores e inconvenientes que generaron retrasos de ejecución y tiempo, éstos se tipifican en diversos items que se muestran a continuación.

#### Metodológicos

- La metodología RAD resultó más eficaz que enfoques tradicionales para un proyecto con componentes de IA, ya que permite validar la viabilidad técnica de integraciones externas (como la API de Gemini) en etapas tempranas, antes de realizar inversiones de tiempo considerables en el desarrollo completo.
- La elaboración detallada del Diagrama de Gantt desde el inicio del proyecto facilitó la identificación temprana de dependencias entre módulos (por ejemplo, el backend debía estar operativo antes de integrar el frontend con síntesis de voz), lo que permitió priorizar correctamente las tareas.

- El levantamiento exhaustivo de requerimientos funcionales y no funcionales al inicio del proyecto evitó cambios de alcance significativos durante la implementación, reduciendo el retrabajo y los conflictos de integración entre los componentes desarrollados en paralelo.

### Uso de herramientas

- El uso de Git con la convención de Conventional Commits y ramas segregadas por tipo (feat/, fix/, docs/) simplificó considerablemente la resolución de conflictos durante el desarrollo paralelo, haciendo el historial de cambios legible y auditable para los tres integrantes del equipo.
- La Web Speech API presentó comportamientos distintos según el navegador y el sistema operativo del dispositivo de prueba, lo que evidenció la importancia de realizar pruebas de compatibilidad cruzada desde etapas tempranas y no asumir comportamiento uniforme entre plataformas.
- La ingeniería de prompts dinámicos resultó ser una disciplina que requiere iteración constante: el ajuste del System Prompt enviado a Google Gemini para delimitar el dominio académico y el tono pedagógico del tutor demandó múltiples ciclos de prueba y refinamiento antes de obtener respuestas consistentes y alineadas con el plan de estudios.

### Calidad

- Las pruebas de validación de la expresión regular para correos institucionales deben incluir casos límite (subdominios, caracteres especiales, dominios similares), ya que los primeros prototipos presentaron falsos positivos que fueron corregidos únicamente al ampliar el conjunto de casos de prueba.
- La separación estricta del código backend en módulos independientes (database.js, gemini.js, rutas REST) facilitó la localización y corrección

de errores, confirmando que la inversión en arquitectura modular desde el inicio reduce significativamente el costo del mantenimiento durante el desarrollo.

- El manejo explícito de excepciones ante fallos de la API de Google Gemini (timeouts, límite de peticiones) demostró ser imprescindible para garantizar una experiencia de usuario aceptable en escenarios de degradación del servicio, aspecto que inicialmente no fue contemplado con suficiente detalle en el diseño.

### Colaborativos

- La comunicación continua entre los integrantes del equipo fue determinante para mantener la coherencia entre el frontend y el backend; las reuniones breves y frecuentes (en lugar de sincronizaciones largas y espaciadas) resultaron más eficaces para detectar incompatibilidades de interfaz de forma temprana.
- La asignación clara de módulos de responsabilidad individual desde el inicio del proyecto redujo la ambigüedad sobre quién debía tomar decisiones técnicas en cada componente, aunque se mantuvo la revisión cruzada de código para garantizar estándares de calidad homogéneos.
- La documentación interna del código (comentarios XML Doc en el backend) facilitó la incorporación y comprensión del trabajo realizado por otros integrantes, evidenciando que documentar durante el desarrollo —y no al final— es una práctica que ahorra tiempo y malentendidos en equipos pequeños.

### PSP — Proceso Personal de Software

El Proceso Personal de Software (PSP), definido por Watts Humphrey en el Software Engineering Institute, es una metodología de mejora continua orientada al

desarrollador individual. Su propósito es que cada programador registre, analice y controle su propio trabajo: el tiempo invertido por tarea, los defectos introducidos y corregidos, y el tamaño del código producido, con el fin de generar datos históricos que permitan realizar estimaciones más precisas y mejorar la calidad del producto de forma sistemática.

En el contexto del presente proyecto, cada integrante del equipo llevó un registro individual de las horas dedicadas a las tareas asignadas, diferenciando entre las fases de diseño, codificación, prueba y corrección de defectos. Este registro permitió contrastar el tiempo planificado en el Diagrama de Gantt con el tiempo real invertido, identificando las actividades que consistentemente demandaron más esfuerzo del estimado. En particular, la integración de la Web Speech API y el ajuste iterativo de los prompts dinámicos para Google Gemini resultaron ser las tareas con mayor desviacón respecto a la estimación inicial.

El análisis PSP también evidenció patrones en la introducción de defectos: la mayoría de los errores detectados durante las pruebas del módulo de autenticación fueron consecuencia de suposiciones incorrectas sobre el comportamiento de la validación Regex en casos límite, errores que podrían haberse evitado con una revisión de código más rigurosa antes de la ejecución de pruebas. Este hallazgo refuerza la práctica de incorporar inspecciones de código como actividad formal dentro del proceso individual de cada desarrollador, reduciendo el costo de corrección al detectar defectos en etapas tempranas del ciclo de vida del software.

Como resultado de la aplicación del PSP, el equipo mejoró progresivamente la precisión de sus estimaciones a lo largo de las iteraciones del proyecto. La comparación entre las métricas de tiempo planificado y tiempo real en cada sprint permitió ajustar la velocidad de desarrollo proyectada para las siguientes fases, contribuyendo a que la entrega final se realizara dentro del margen de tiempo establecido en la planificación del proyecto.

## TSP — Proceso de Software en Equipo

El Proceso de Software en Equipo (TSP), también desarrollado por Watts Humphrey como extensión del PSP al contexto grupal, establece un marco estructurado para que equipos pequeños de desarrollo planifiquen, coordinen y ejecuten su trabajo de manera disciplinada. El TSP parte de los datos individuales recopilados mediante el PSP y los integra en un plan de equipo cohesionado, definiendo roles específicos, estrategias de mitigación de riesgos y criterios de calidad colectivos.

En el equipo del Tutor Virtual UMAD, conformado por tres desarrolladores, se asignaron roles complementarios al inicio del proyecto: un Líder de Equipo responsable de la coordinación general, la comunicación con el director de tesis y el seguimiento del Gantt; un Líder de Calidad encargado de establecer los estándares de codificación, coordinar las revisiones cruzadas de código y verificar el cumplimiento de los requerimientos no funcionales; y un Líder de Desarrollo responsable de las decisiones arquitectónicas, la integración de componentes y la resolución de conflictos técnicos entre módulos. Esta estructura de roles, inspirada en el TSP, proporcionó claridad sobre las responsabilidades de cada integrante sin eliminar la colaboración transversal entre ellos.

La planificación de equipo al inicio de cada iteración siguió el ciclo del TSP: lanzamiento (definición de objetivos del sprint), estrategia (selección de requerimientos a implementar), plan (estimación de tiempos por tarea y asignación), ejecución (desarrollo con registro PSP individual) y análisis postmortem (revisión de desviaciones y definición de acciones de mejora). Este ciclo, aunque adaptado informalmente a la escala del proyecto académico, resultó efectivo para mantener la alineación del equipo y detectar oportunamente los riesgos técnicos antes de que se convirtieran en problemas de integración.

Uno de los beneficios más concretos de aplicar principios TSP fue la gestión explícita de la deuda técnica: cuando por restricciones de tiempo se tomó la

decisión de implementar una solución provisional en el manejo de sesiones concurrentes, esta decisión fue documentada formalmente como un ítem pendiente de refactorización, garantizando su visibilidad para el equipo y su consideración en las fases futuras de desarrollo. Esta práctica de transparencia sobre las decisiones de diseño apresuradas es un elemento central del TSP y resultó valiosa para mantener la integridad conceptual del sistema a lo largo de las iteraciones.

## 4.2. Recomendaciones

A continuación se listan las recomendaciones propuestas para ampliar y robustecer el presente proyecto:

- Migrar la base de datos de SQLite a un motor cliente-servidor como PostgreSQL o MySQL para soportar concurrencia masiva de usuarios en caso de una adopción institucional amplia que trascienda el grupo inicial de estudiantes de Ingeniería de Software.
- Expandir el sistema de enrutamiento de prompts para abarcar los planes de estudios de otras carreras ofertadas por la UMAD (Derecho, Administración, Psicología), reutilizando la arquitectura existente y añadiendo únicamente los prompts especializados correspondientes a cada programa académico.
- Implementar autenticación OAuth 2.0 con Google Workspace institucional, eliminando la necesidad de gestionar contraseñas locales y aprovechando la infraestructura de identidad ya existente en la universidad para simplificar el proceso de registro y acceso.
- Desarrollar un panel de administración para docentes que les permita cargar materiales propios (apuntes, guías, ejercicios) como contexto adicional para el modelo de IA, incrementando la alineación entre las respuestas del tutor y los contenidos específicos impartidos en cada asignatura.
- Incorporar métricas de aprendizaje del alumno mediante el análisis semántico de las consultas históricas, identificando patrones de confusión recurrentes y generando reportes periódicos que los docentes puedan consultar para ajustar su práctica pedagógica.
- Explorar la migración del avatar 2D hacia una representación 3D con animaciones más naturales, utilizando tecnologías web como Three.js o bibliotecas especializadas de síntesis facial, para incrementar el engagement del usuario y el efecto de presencia de un tutor humano virtual.

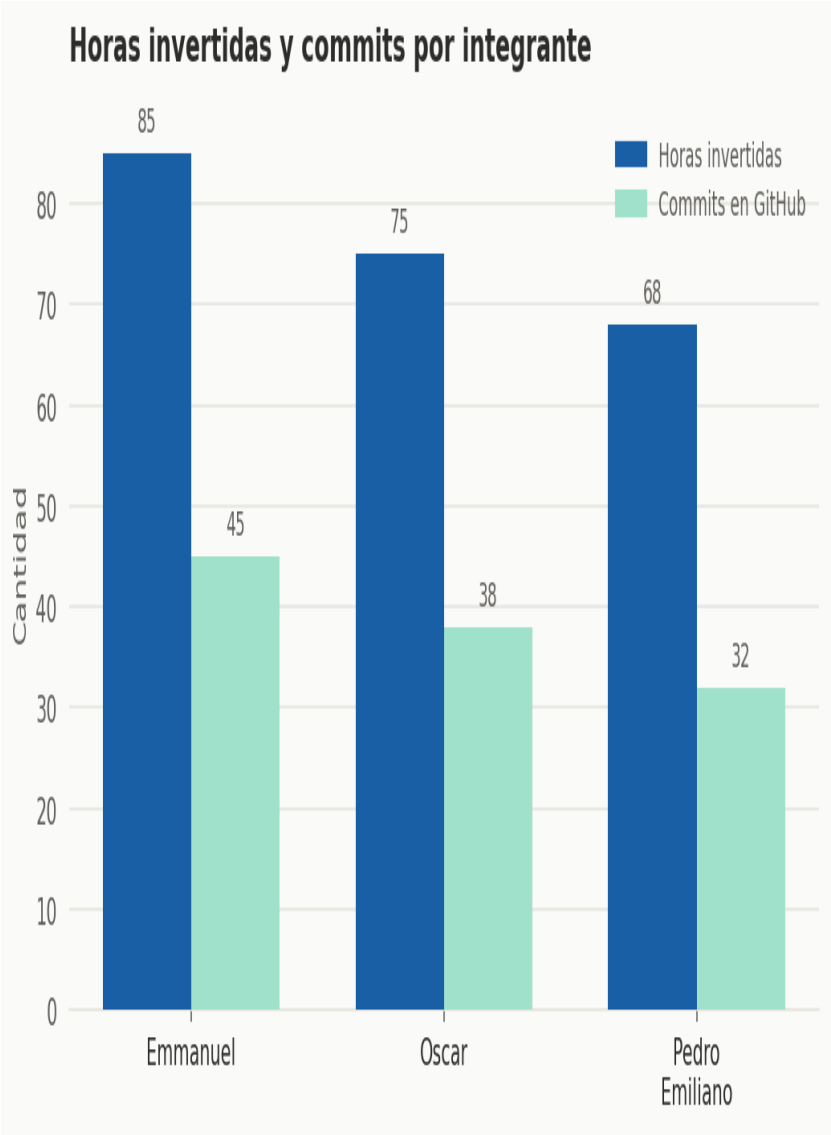



**Figura 4.1:** Esfuerzo técnico y entregables por integrante del equipo.

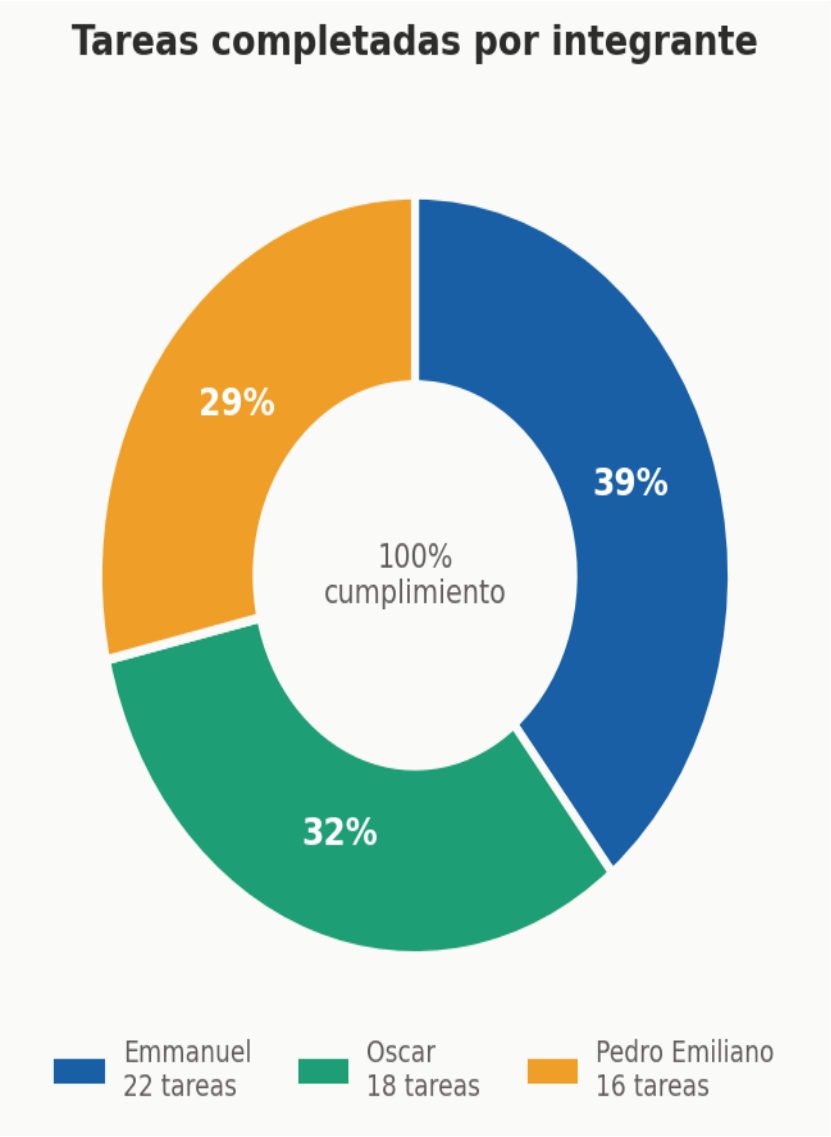



**Figura 4.2:** Esfuerzo técnico y entregables por integrante del equipo.

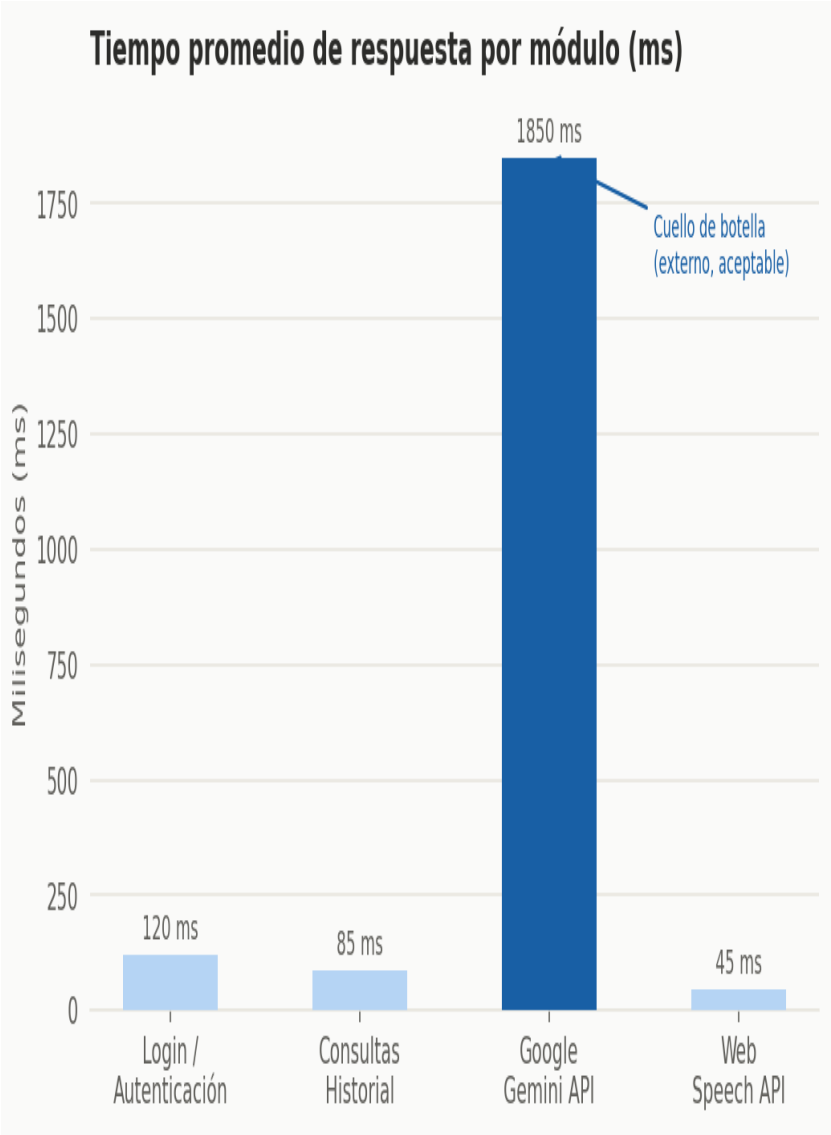



**Figura 4.3:** Tiempos de respuesta (latencia) por módulo del sistema.

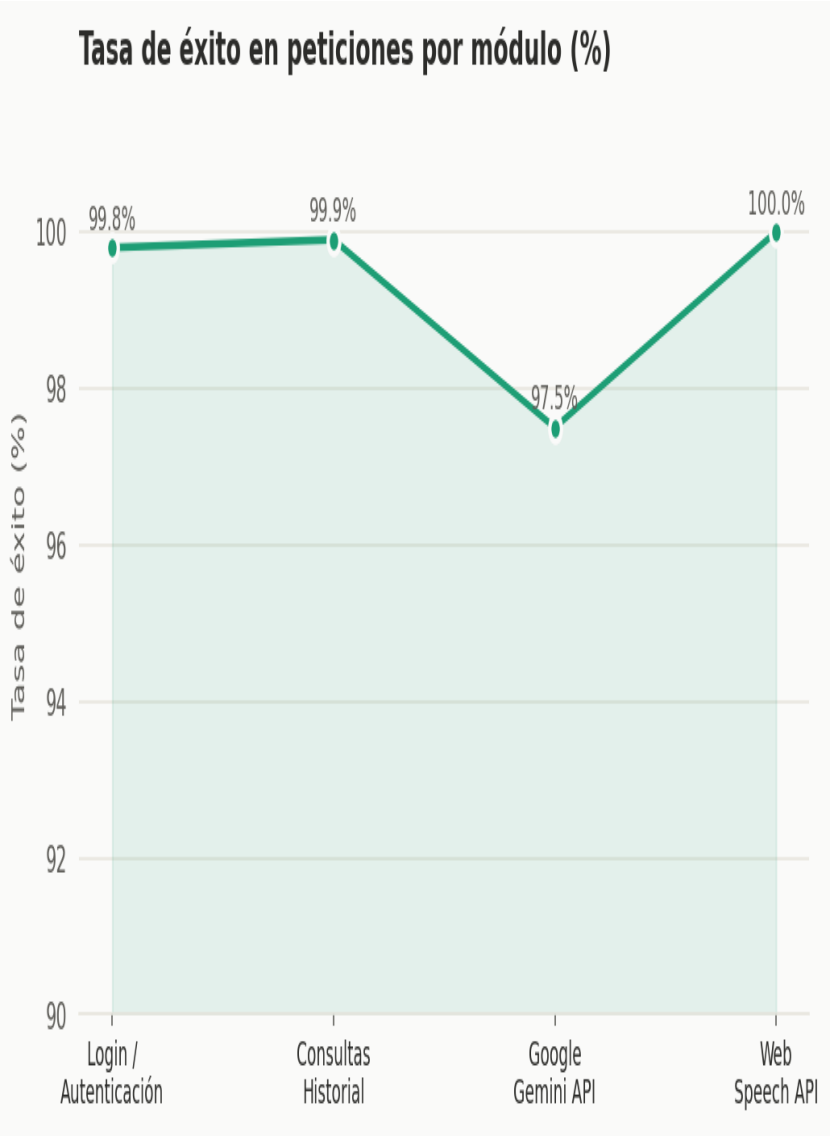



**Figura 4.4:** Tiempos de respuesta (latencia) por módulo del sistema.

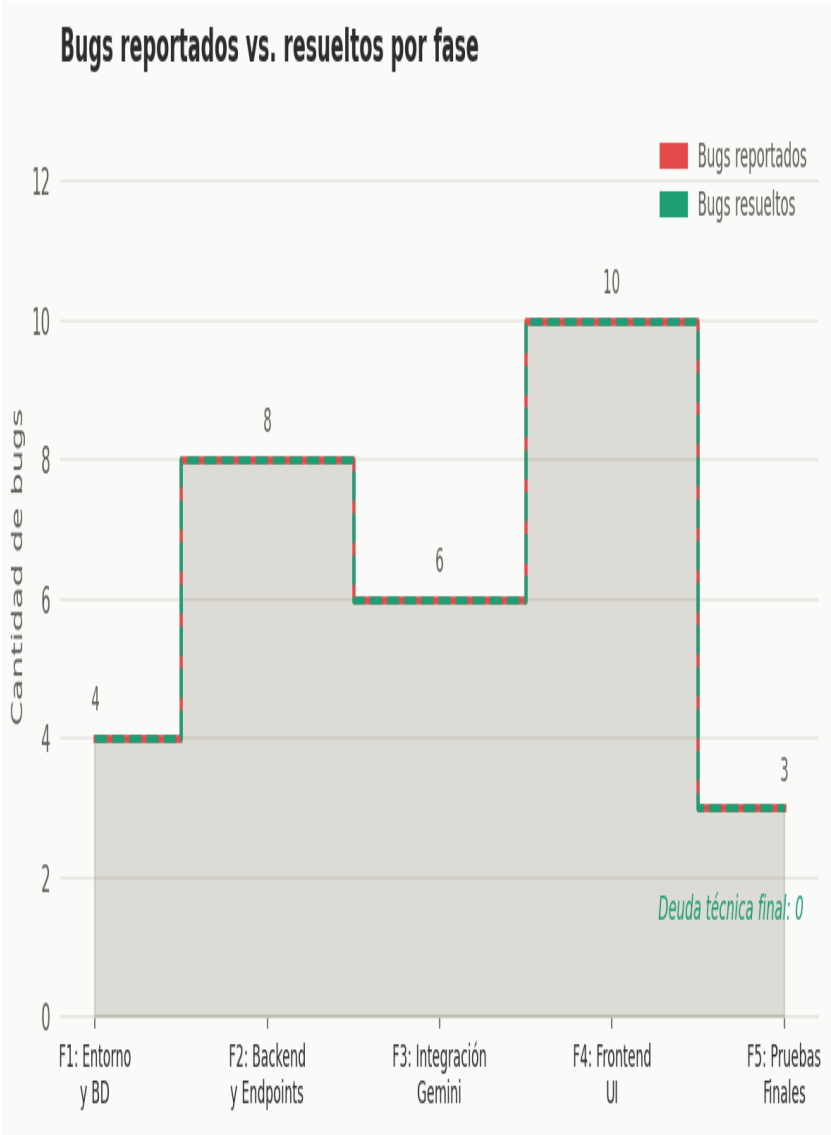



**Figura 4.5:** Curva de resolución de bugs a lo largo de las fases del proyecto.

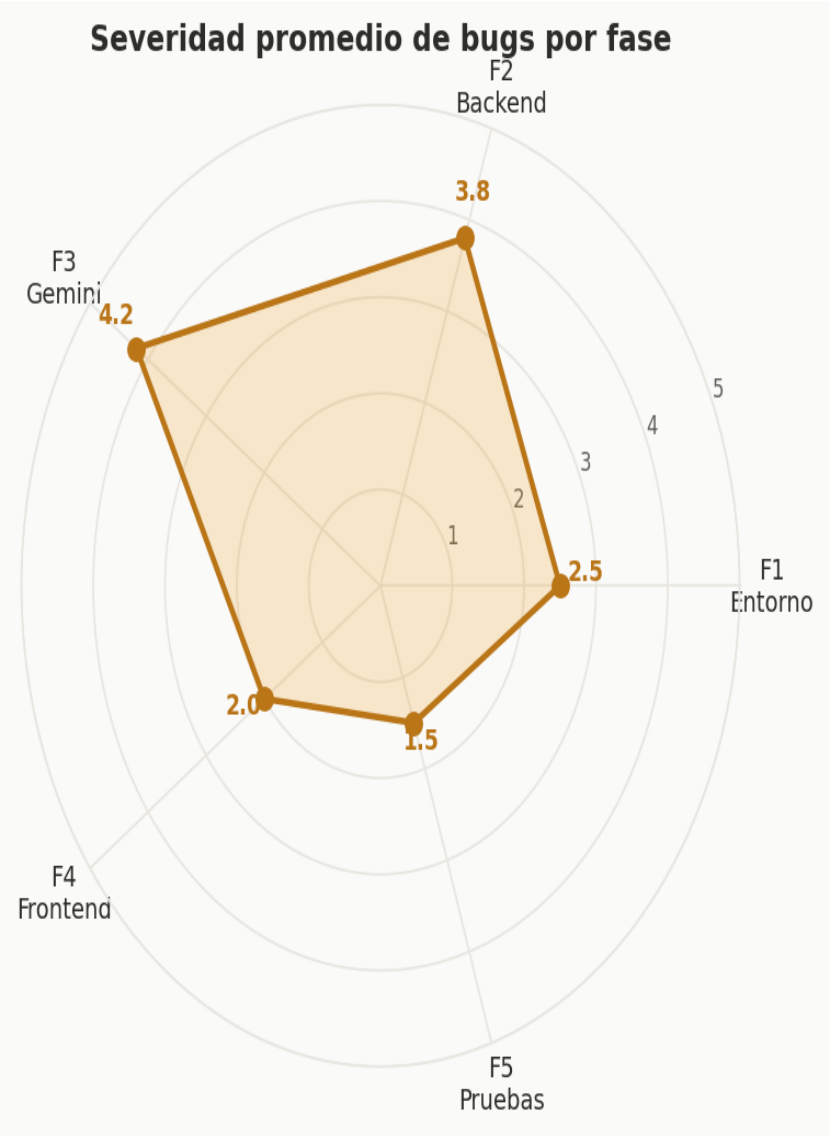



**Figura 4.6:** Curva de resolución de bugs a lo largo de las fases del proyecto.

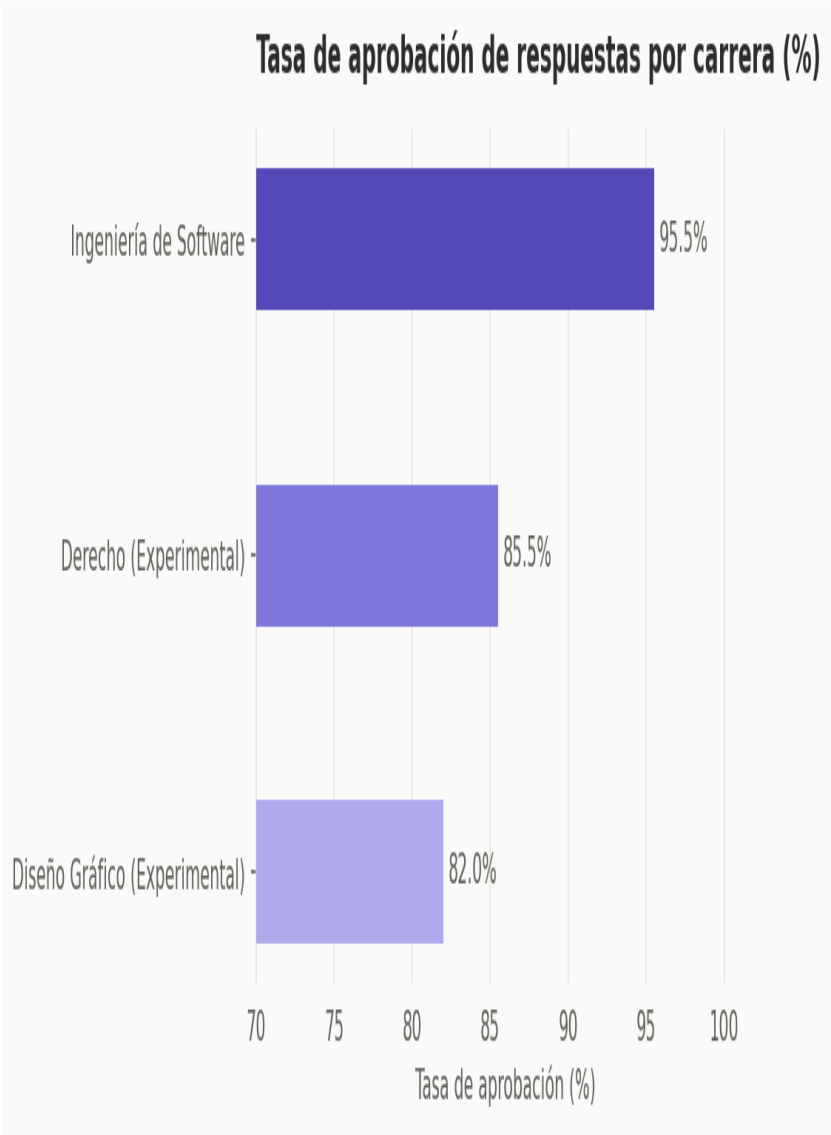



**Figura 4.7:** Tasa de aprobación del sistema por perfil de usuario simulado.

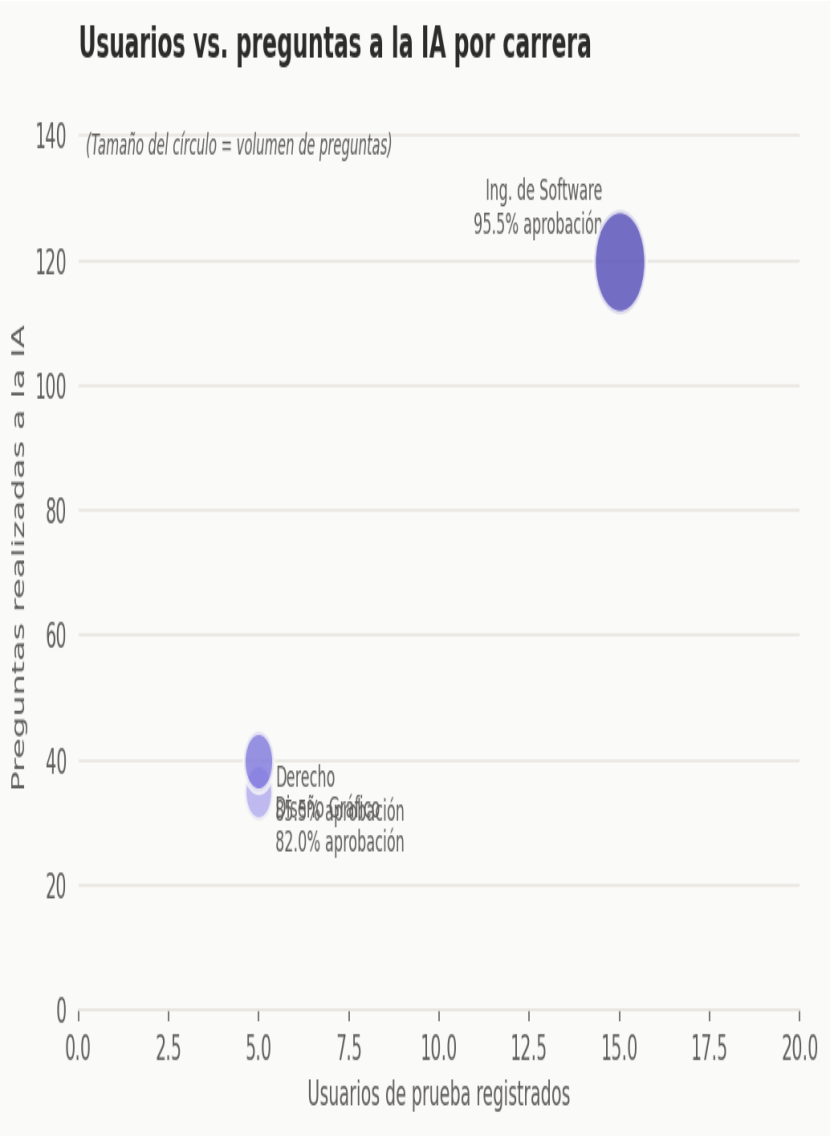
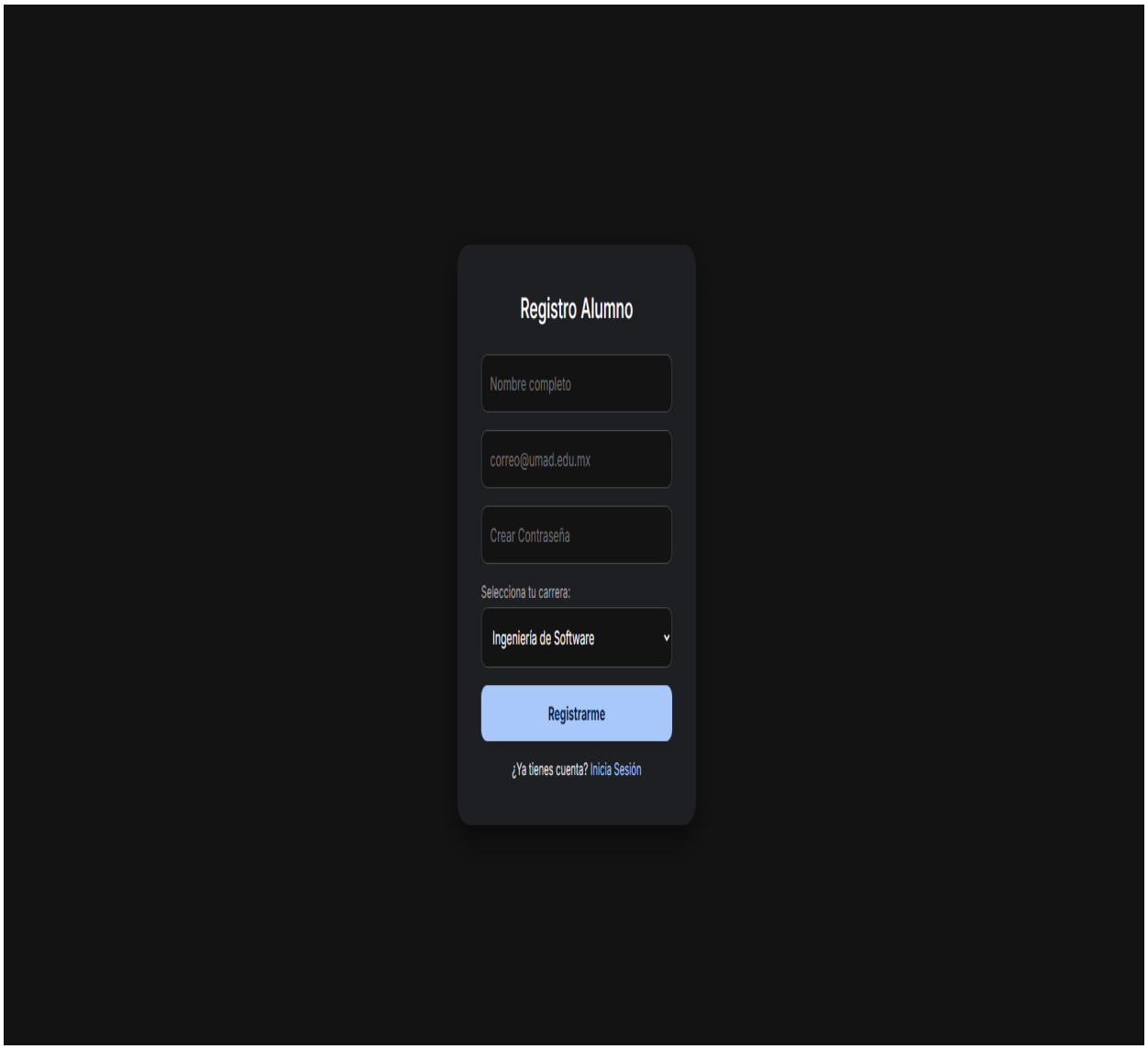
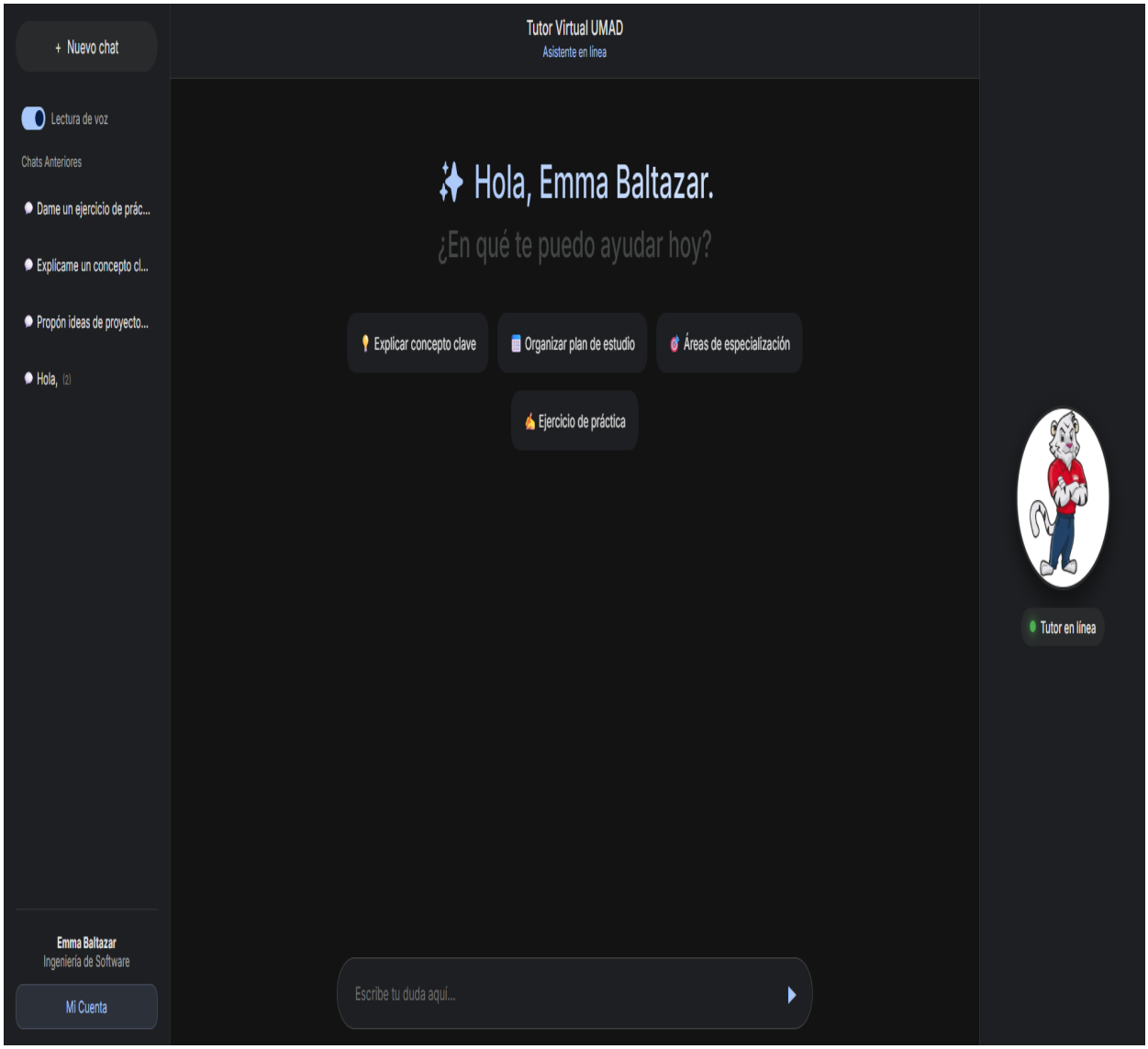



**Figura 4.8:** Tasa de aprobación del sistema por perfil de usuario simulado.


Registro Alumno
Nombre completo
correo@umad.edu.mx
Crear Contraseña
Selecciona tu carrera:
Ingeniería de Software
Registrarme
¿Ya tienes cuenta? Inicia Sesión


+ Nuevo chat
Lectura de voz
Chats Anteriores
Dame un ejercicio de prác...
Explícame un concepto cl...
Propón ideas de proyecto...
Hola,
Emma Baltazar
Ingeniería de Software
Mi Cuenta
Tutor Virtual UMAD
Asistente en línea
Hola, Emma Baltazar.
¿En qué te puedo ayudar hoy?
Explicar concepto clave
Organizar plan de estudio
Áreas de especialización
Ejercicio de práctica
Escribe tu duda aquí...
Tutor en línea

# Referencias


[1] OpenAI, “Teaching with ai,” 2023.

[2] UNESCO, “Guía para el uso de ia generativa en educación e investigación,” 2024.

[3] M. W. Docs, “Api web de voz,” s.f.

[4] W. C. Group, “Web speech api,” 2025.

[5] Node.js, “Sobre node.js,” s.f.

[6] W3C, “Web standards,” s.f.

[7] J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, “Cognitive tutors: Lessons learned. journal of the learning sciences,” 1995.

[8] C. Piech *et al.*, “Deep knowledge tracing,” 2015.

[9] UNESCO, “Guidance for generative ai in education and research,” 2023.

[10] OECD, “Oecd principles on artificial intelligence,” 2019.

[11] E. Commission, “Ai act,” 2024.

[12] N. N. Group, “10 usability heuristics for user interface design,” 2020.

[13] W3C, “Web content accessibility guidelines (wcag) 2.1,” 2018.